

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Mobile Money Investigations with PostgreSQL and Neo4j

Advanced Big Data Analytics: Project 2

Student Name	Student ID
RWAGASANA Felicien	101364
RUBAGUMYA BRAVE	101392
UMUHOZA Diane	101286

Program: MSc in Big Data Analytics
Course: Advanced Big Data Analytics
Lecturer: Temitope Oguntade
GitHub: https://github.com/RwagasanaFelc/Advanced-Big-Data-Analytics-Project-2
Date: October 26, 2026

Contents

Introduction 3
1 Capture the Data 3
1.1 Prepare the Workspace 3
1.2 Build capture.py 5
2 Build PostgreSQL 8
2.1 Define the Five Tables 8
2.2 Bulk-load and Check 10
3 Build Neo4j 13
3.1 Map the Same Facts 13
3.2 Prepare and Import CSVs 14
3.3 Inspect One Account in Both Models 17
4 Answer Routine Business Questions 18
4.1 Q1 Account Activity 18
4.2 Q2 Daily Activity 19
4.3 Q3 Shared Devices 21
5 Investigate Connected Accounts 22
5.1 Q4 Ownership and Device Network 22
5.2 Q5 Rapid Onward Payments 24
5.3 Q6 Circular Payments 25
5.4 Q7 Shortest Transfer Connection 27
6 Check Correctness and Performance 29
6.1 Build One Comparison Program 29
6.2 Time a Fixed Selection 30
6.3 Try One SQL Improvement 36
7 Test Concurrent Requests 38
7.1 Build a Small Load Test 38
7.2 Explain Scaling 40
8 Make and Defend Decisions 41
8.1 Prioritise Investigation 41
8.2 Recommend a Model 43
8.3 Explain Individual Judgement 44
9 Submit Reproducible Work 46
9.2 Provide Code and Verification 46

Introduction

A mobile money provider in Rwanda wants its analysts to explain payment activity and to identify accounts that need closer review. Daily reports show how much money moves through the service, but an investigation has to follow transfers between wallets and examine who owns those wallets and which devices they use.

This report builds the same reports and investigations twice: in **PostgreSQL 16** (relational model) and in **Neo4j 5 Community** (graph model). Both hold the same 20,993,500 September 2026 records captured from the supplied, unchanged generator `mobile_money_generator.py`. For each question we compare how the two models express the problem, prove that both return identical answers (61 of 61 cases), measure single-query speed and concurrent load, and recommend which model suits each task. All times are Central Africa Time (CAT, Africa/Kigali).

Set	Review account	Destination (Q7)	September day
A	998	24755	6
B	147228	186823	13
C	206978	6573	8
D	237998	21755	6
E	144228	183823	13
F	203978	3573	8

Table 1: Review sets; the hour is 09:45:00 (inclusive) to 10:45:00 (exclusive) CAT

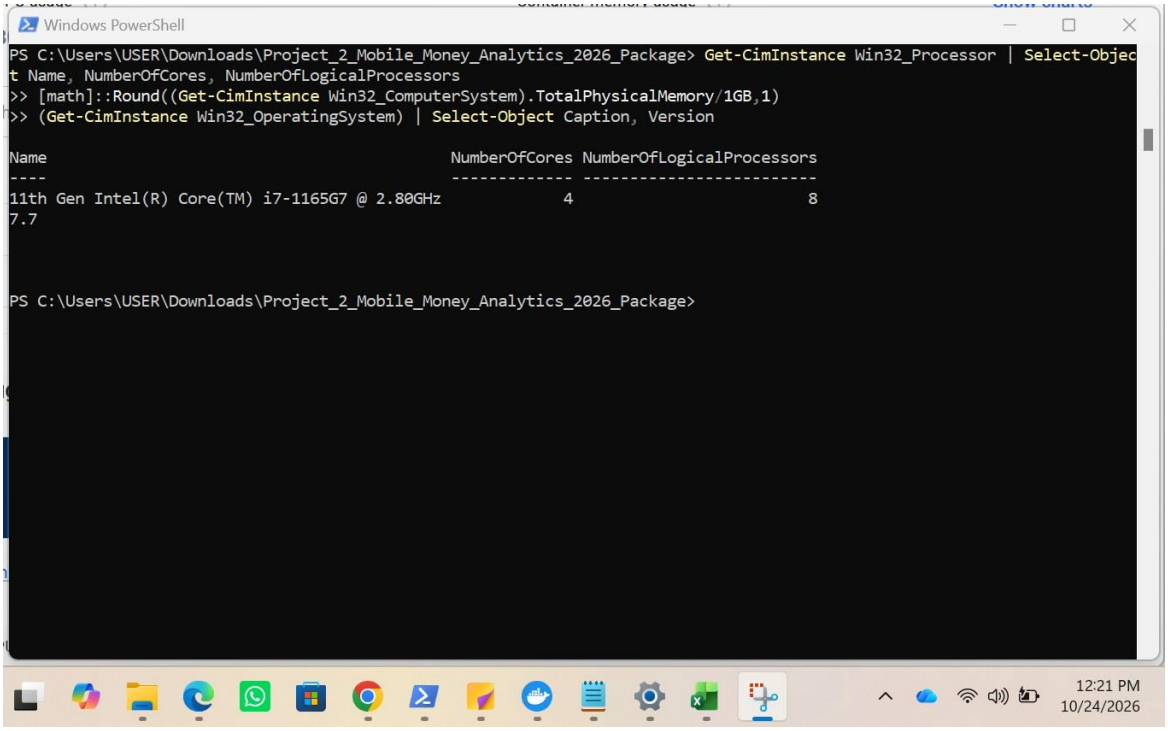
1 Capture the Data

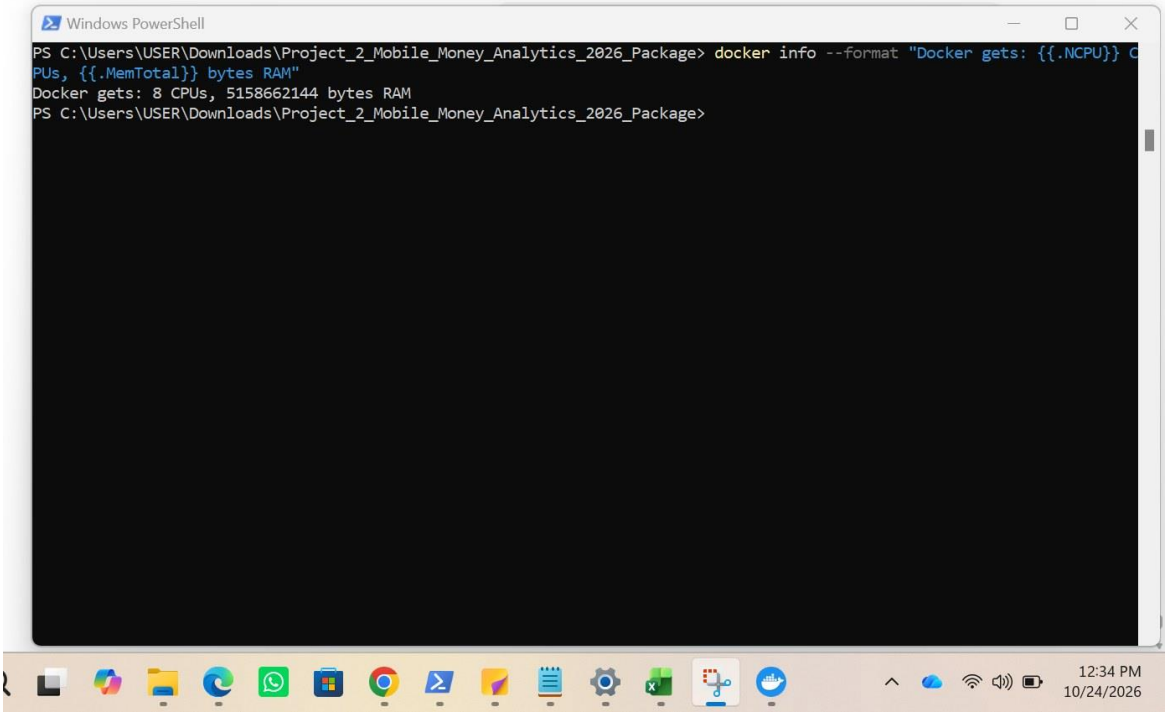
1.1 Prepare the Workspace

Both databases run in Docker on one machine, each limited to 4 CPUs and 2.5 GB of memory, so that neither engine gets more resources than the other.

Item	Value
Operating system	Microsoft Windows 11 Pro (version 10.0.22631)
CPU	Intel Core i7-1165G7 @ 2.80 GHz, 4 cores, 8 logical CPUs
RAM	8.3 GB (8 GB installed)
Docker	Docker Engine 29.9.2 (Docker Desktop VM: 8 CPUs, 5.16 GB); each container limited to 4 CPUs and 2.5 GB
PostgreSQL	16.15, database mobilemoney, timezone = Africa/Kigali
PostgreSQL memory	shared_buffers 1 GB, effective_cache_size 2 GB, work_mem 32 MB, maintenance_work_mem 256 MB, max_parallel_workers_per_gather 2
Neo4j	5.26.32 Community, default neo4j database
Neo4j memory	heap 1 GB (initial = max), page cache 1 GB
Python and drivers	Python 3.12.10, psycopg2 2.9.13, neo4j Python driver 6.4.1

Table 2: Software, hardware and database memory settings


Windows PowerShell
PS C:\Users\USER\Downloads\Project_2_Mobile_Money_Analytics_2026_Package> Get-CimInstance Win32_Processor | Select-Objec
t Name, NumberOfCores, NumberOfLogicalProcessors
>> [math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory/1GB,1)
>> (Get-CimInstance Win32_OperatingSystem) | Select-Object Caption, Version
Name NumberOfCores NumberOfLogicalProcessors
11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz 4 8
7.7
PS C:\Users\USER\Downloads\Project_2_Mobile_Money_Analytics_2026_Package>
12:21 PM
10/24/2026


Windows PowerShell
PS C:\Users\USER\Downloads\Project_2_Mobile_Money_Analytics_2026_Package> docker info --format "Docker gets: {{.NCPU}} C
PUs, {{.MemTotal}} bytes RAM"
Docker gets: 8 CPUs, 5158662144 bytes RAM
PS C:\Users\USER\Downloads\Project_2_Mobile_Money_Analytics_2026_Package>
12:34 PM
10/24/2026

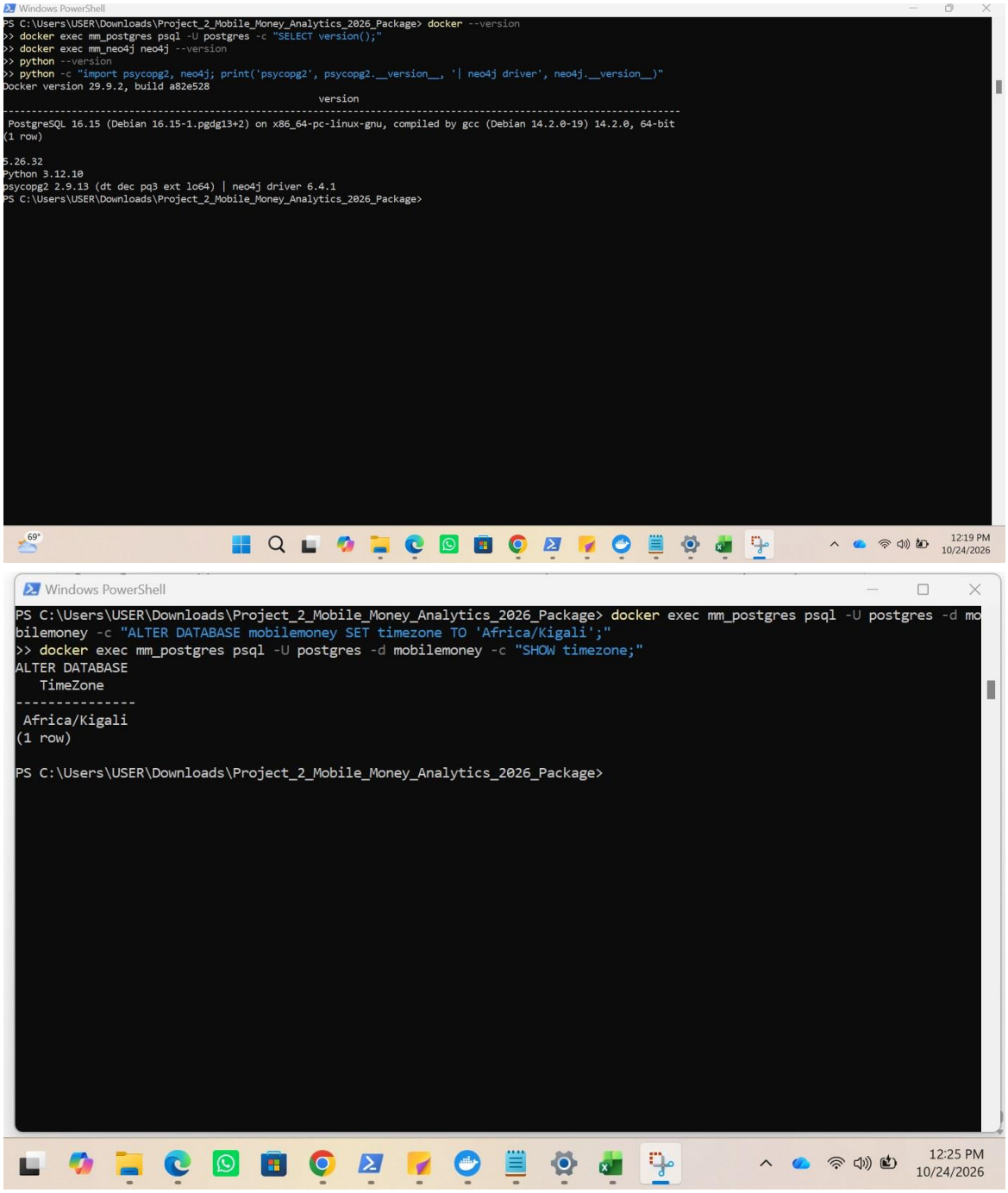


Figure 1: Hardware and OS, CPU and RAM given to Docker, software and driver versions, and the CAT time zone setting

The first screenshots show the laptop (Intel Core i7-1165G7, 4 cores and 8 threads, 7.7 GiB = 8.3 GB of RAM) and the 8 CPUs and 5.16 GB that Docker can use. The next shows PostgreSQL 16.15, Neo4j 5.26.32, Python 3.12.10 and the driver versions, and the last sets and confirms the database time zone Africa/Kigali.

1.2 Build capture.py

`capture.py` imports the generator as g, inspects `g.dataset_info()` and `g.FIELDS`, and previews ten records from each `iter_<stream>()` with `itertools.islice`. It then writes each stream progressively with `csv.DictWriter`, counting consumed and written rows and logging every million transfers. For transfers it also accumulates the count, the earliest and latest `occurred_at` and the sum of `amount_rwf`. No stream is ever collected in memory.

Stream	Rows written	Capture (s)	CSV size (GB)
customers	228,000	0.3	0.0036
accounts	248,000	0.4	0.0052
devices	228,600	0.4	0.0036
account_devices	285,600	0.8	0.0063
transfers	20,003,300	156.1	0.7802
Total	20,993,500	158.0	0.7988

Table 3: Captured streams (consumed = written for every stream)

Transfer audit totals: 20,003,300 transfers, earliest `occurred_at` 1788235200 (1 September 2026, 06:00:00 CAT), latest 1790798399 (30 September 2026, 21:59:59 CAT), total value 231,482,327,870 RWF.

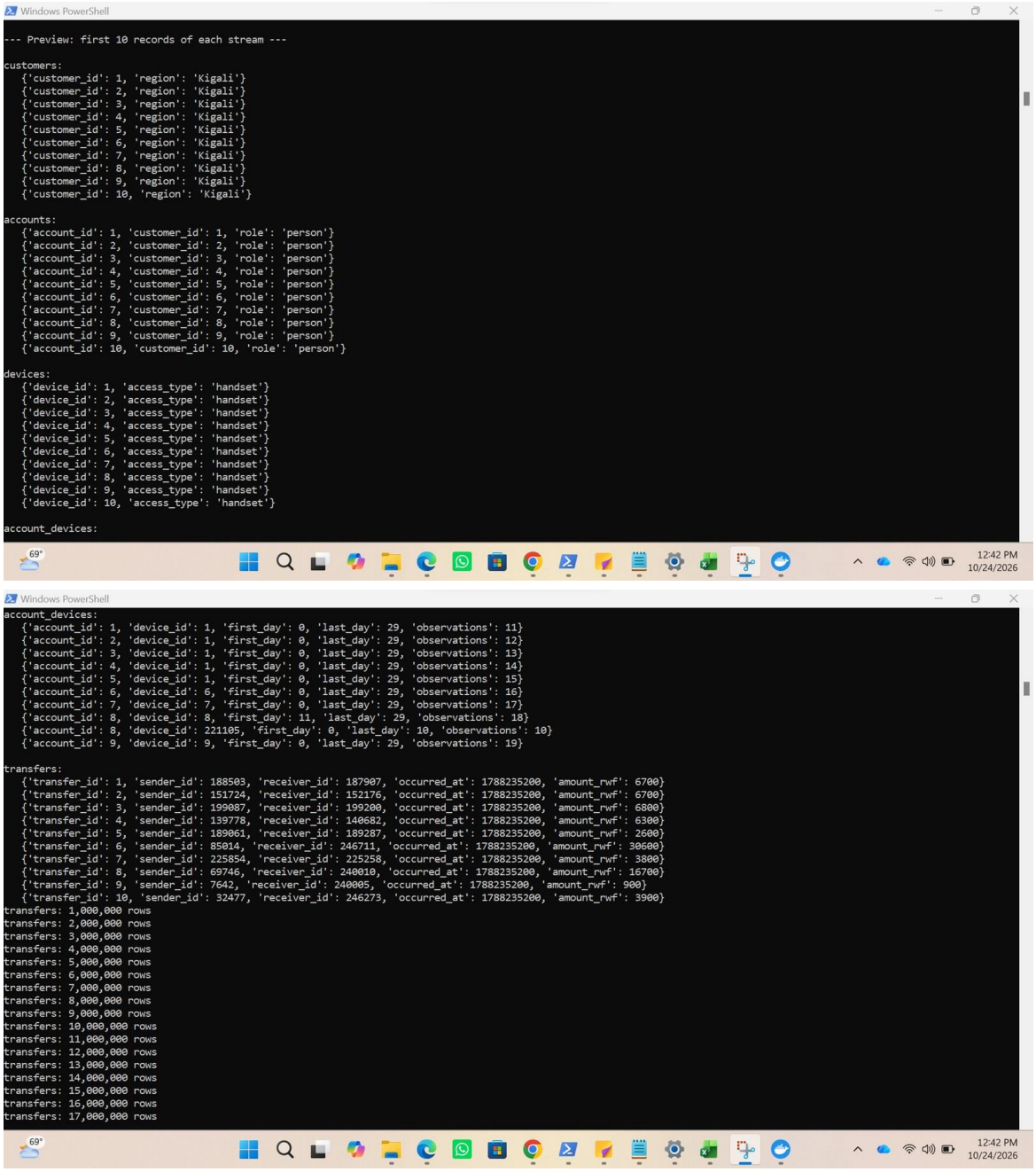


Figure 2: Preview of ten records from each stream

The first ten records of each stream, read with `itertools.islice` from a fresh iterator, with fields in `g.FIELDS` order. The second screenshot continues with account_devices and transfers and shows the progress line logged every million transfers.

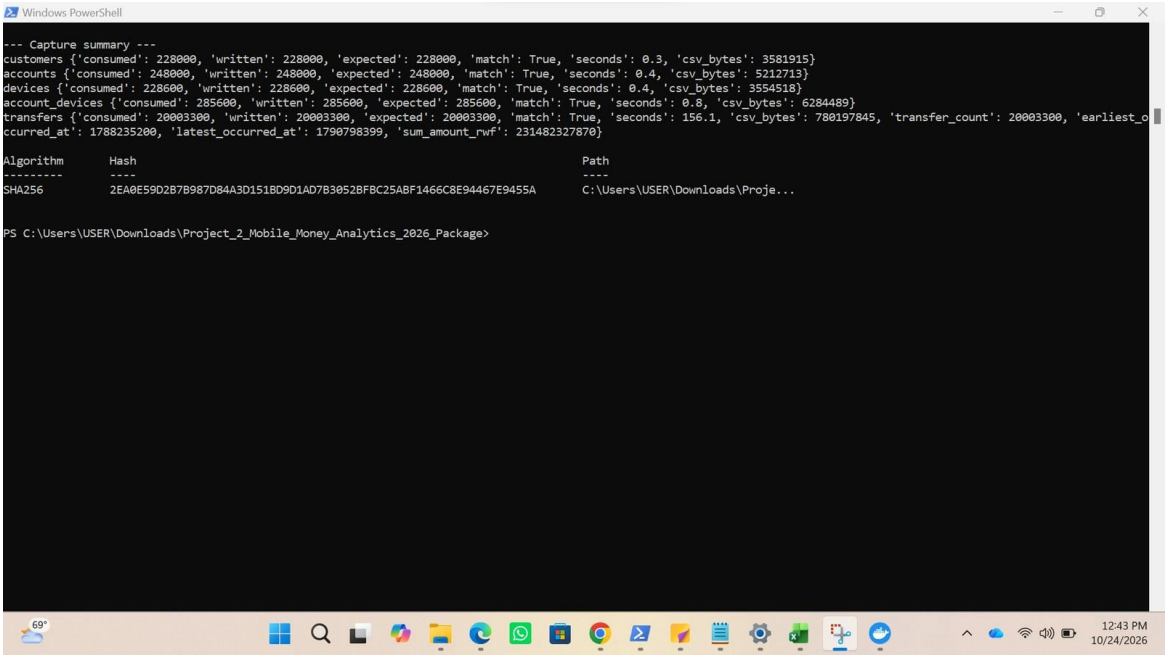


Figure 3: Completed five-stream totals and transfer audit totals

For every stream consumed = written = expected. Capture took 158 s in total, almost all for the 20,003,300 transfers, and the transfer audit (count, earliest and latest occurred_at, total RWF) is printed for later reconciliation.

2 Build PostgreSQL

2.1 Define the Five Tables

The five tables follow the captured CSVs exactly. Entity identifiers, amounts and observations are `INTEGER`; `transfer_id` and `occurred_at` are `BIGINT`; observation days are `SMALLINT`; categories are `TEXT`. Every column is `NOT NULL`.

Table	Key	Fields, meaning and units
customers	customer_id	region: customer's home region (Kigali, Eastern, Northern, Southern, Western)
accounts	account_id	customer_id: owner; role: person, merchant or agent
devices	device_id	access_type: handset or public terminal
account_devices	(account_id, device_id)	first_day, last_day: inclusive day numbers 0–29 from 1 September; observations: count of observations in that interval
transfers	transfer_id	sender_id, receiver_id: accounts; occurred_at: Unix seconds; amount_rwf: whole Rwandan francs

Table 4: Tables, keys and field meanings

Foreign keys link ownership (`accounts.customer_id`), device use (both columns of `account_devices`) and both transfer endpoints to their parent tables. Check constraints require `amount_rwf > 0`, `observations > 0` and `0 <= first_day <= last_day <= 29`.

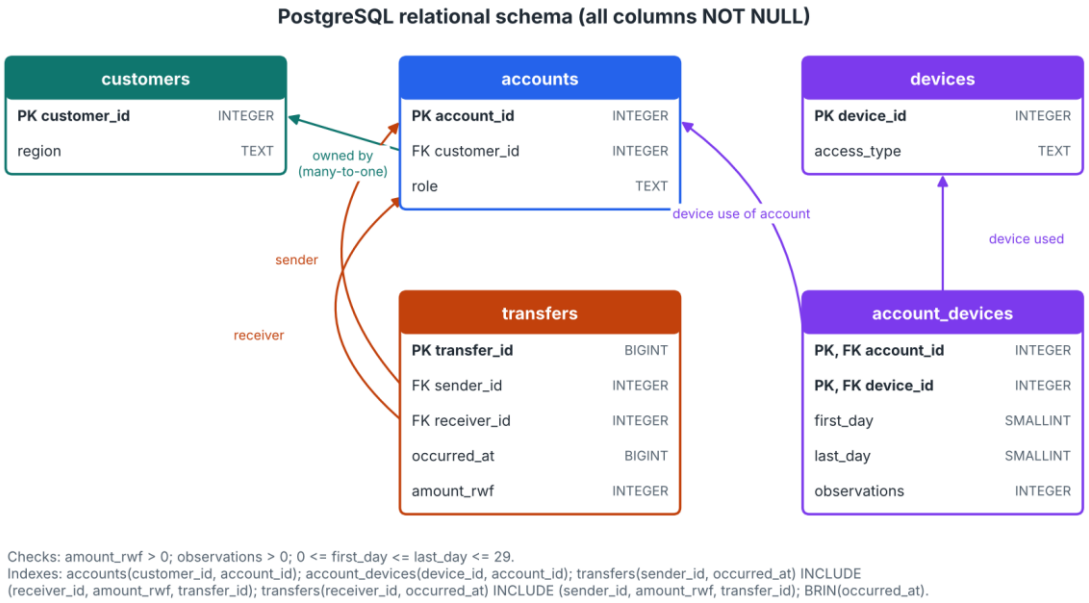


Figure 4: PostgreSQL schema

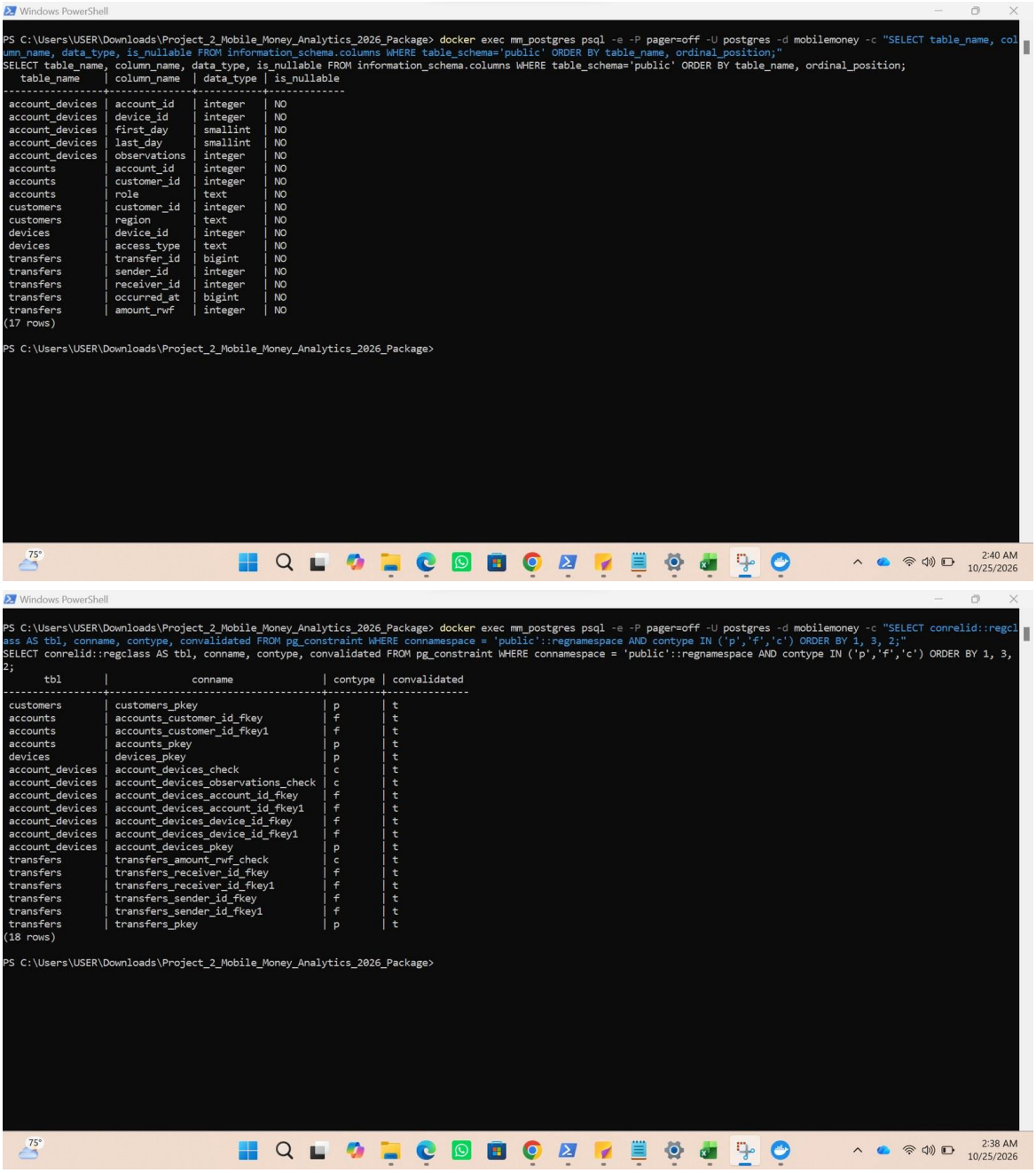


Figure 5: Table definitions and validated constraints

The column listing shows every column with its type and `is_nullable = NO`. The constraint listing shows primary keys (p), foreign keys (f) and checks (c), all validated (t); the duplicate `fkey1` names come from running `pg_02` twice.

2.2 Bulk-load and Check

Customers and devices were loaded first, then accounts, account_devices and transfers, with `\copy … CSV HEADER` (`pg_01_schema_load.sql`). Constraints and indexes were added afterwards (`pg_02_constraints_indexes.sql`): composite indexes on `accounts(customer_id, account_id)` and `account_devices(device_id, account_id)`, covering transfer indexes on `(sender_id, occurred_at)` and `(receiver_id, occurred_at)`, a BRIN index on `occurred_at`, then `ANALYZE`.

Measure	Value
Loading time (five \copy commands)	32.4 s
Constraint and index setup time (before ANALYZE)	43.2 s
Table data size	1.25 GB
Index size	2.47 GB
Total database size	3.73 GB

Table 5: PostgreSQL load and storage (1 GB = 1,000,000,000 bytes)

All row counts and the transfer audit totals match `capture.py`, and the roles are 240,000 person, 6,000 merchant and 2,000 agent accounts. `pg_02` was accidentally run twice, which created duplicate foreign keys (suffix `fkey1` in the constraint listing) and duplicate indexes; the duplicate indexes were dropped before sizes and timings were measured, leaving the 10 indexes shown.

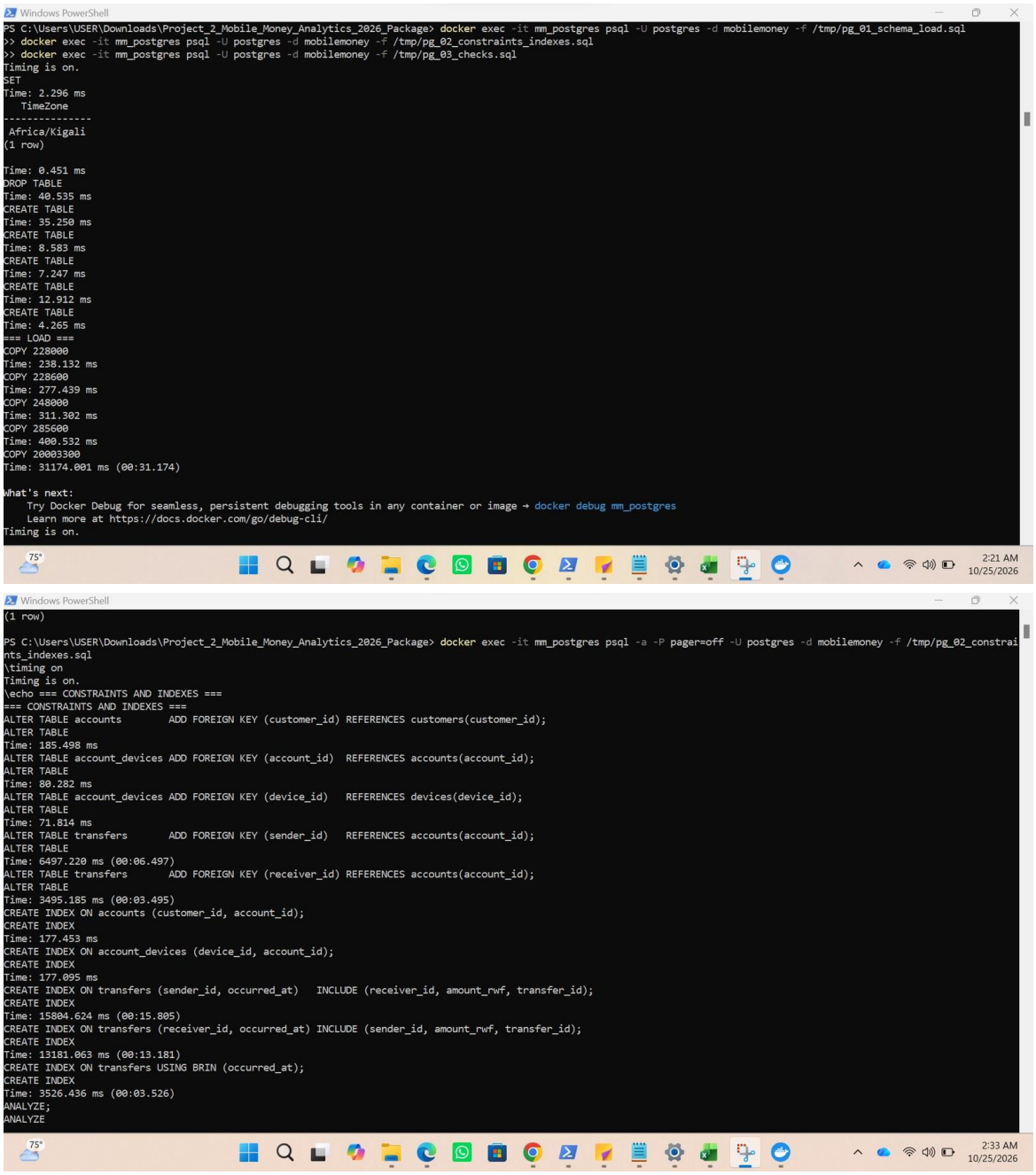


Figure 6: Load timing (\copy) and constraint/index timing

The first screenshot shows the five \copy commands with their times (31.2 s for transfers, 32.4 s in total). The second shows each foreign key and index with its time (43.2 s in total); the two covering transfer indexes are the slowest (15.8 s and 13.2 s).

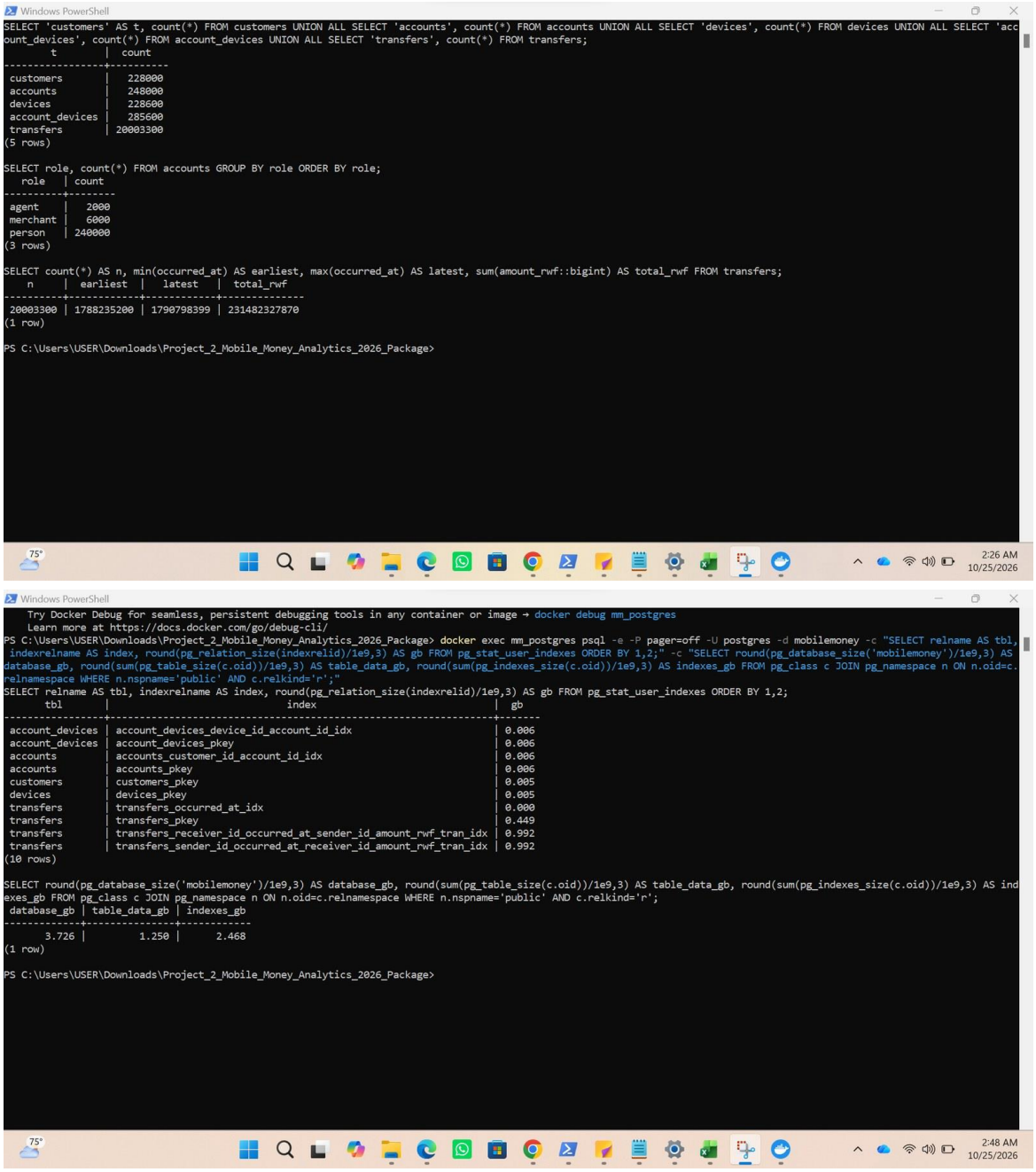


Figure 7: Row counts, roles, audit totals, validated constraints, indexes and sizes

Row counts equal the captured counts, the roles are 2,000 agent, 6,000 merchant and 240,000 person, and the transfer audit matches (20,003,300 transfers, 1788235200 to 1790798399, 231,482,327,870 RWF). The second screenshot lists the 10 indexes and the sizes: 3.726 GB database, 1.250 GB tables and 2.468 GB indexes.

3 Build Neo4j

3.1 Map the Same Facts

Source field	Graph element
customers.customer_id, region	(:Customer {customer_id, region})
accounts.account_id, role	(:Account {account_id, role})

Source field	Graph element
accounts.customer_id	(:Customer)-[:OWNS]->(:Account), from owns.csv
devices.device_id, access_type	(:Device {device_id, access_type})
account_devices (all fields)	(:Account)-[:USED_DEVICE {first_day, last_day, observations}]->(:Device)
transfers (all fields)	(:Account)-[:TRANSFER {transfer_id, occurred_at, amount_rwf}]->(:Account), one per payment

Table 6: Mapping of every source field to the graph

Every transfer is its own relationship, so repeated payments between the same two accounts are preserved.

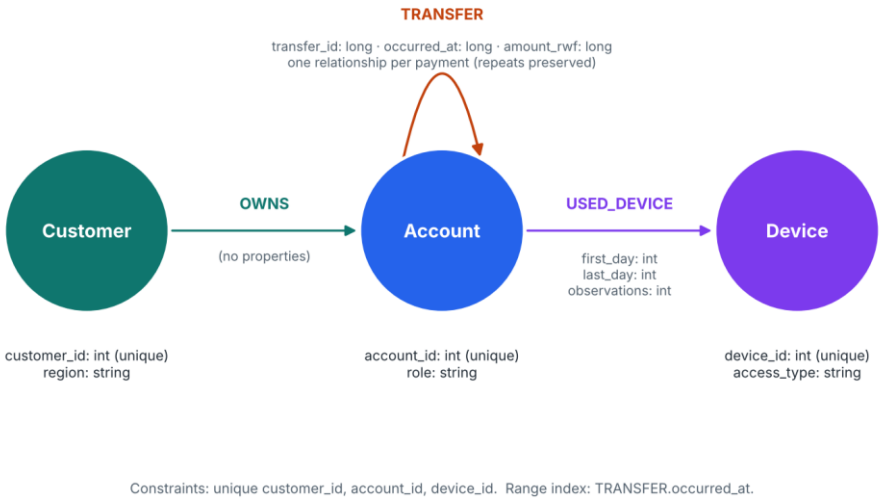


Figure 8: Neo4j graph schema

3.2 Prepare and Import CSVs

`prepare_import.py` writes temporary copies of the CSVs into `import/` with typed headers, leaving the source CSVs unchanged, and derives `owns.csv` in (customer_id, account_id) order. The accounts header is `account_id:ID(Account),:IGNORE,role:string`; separate ID groups keep Customer 7, Account 7 and Device 7 distinct. The import ran with the database stopped:

```
neo4j-admin database import full neo4j --overwrite-destination --id-type=integer
  --nodes=Customer=customers.csv --nodes=Account=accounts.csv --nodes=Device=devices.csv
  --relationships=OWNS=owns.csv --relationships=USED_DEVICE=account_devices.csv
  --relationships=TRANSFER=transfers.csv
```

After start-up, unique constraints were created on `customer_id`, `account_id` and `device_id`, plus a range index on `TRANSFER.occurred_at`, and we waited for them to be online.

Measure	Value
Import time (IMPORT DONE; 120.9 s with container start)	109.3 s
Constraint and index setup time	44.7 s
Store size	2.17 GB
Transaction-log size	0.27 GB

Table 7: Neo4j import and storage

Checks: 704,600 nodes (228,000 Customer, 248,000 Account, 228,600 Device) and 20,536,900 relationships (OWNS 248,000, USED_DEVICE 285,600, TRANSFER 20,003,300); roles as in PostgreSQL; every account has exactly one owner; `transfer_id` is unique (0 duplicates); the audit totals match the capture. The temporary import copies were then removed.

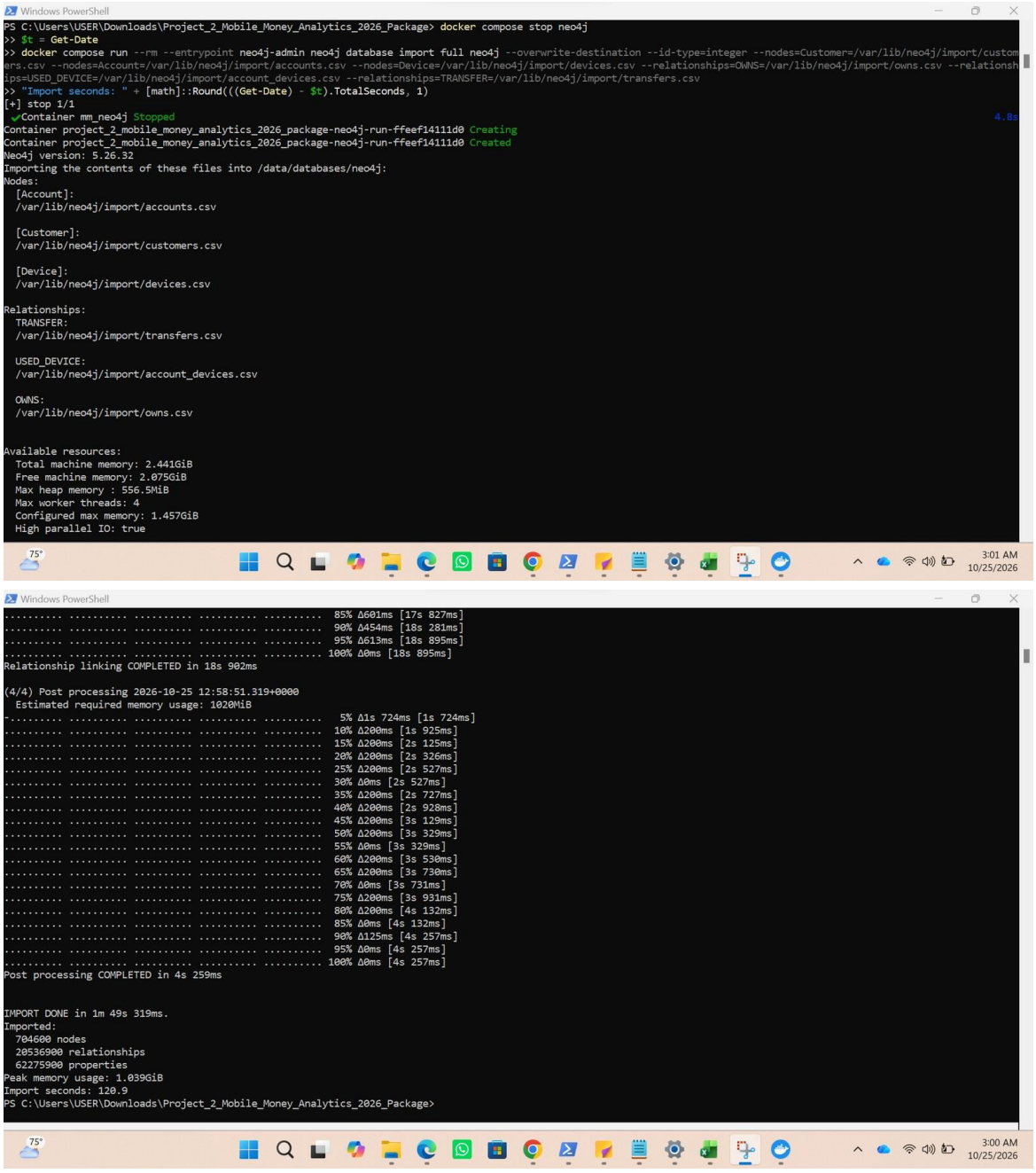


Figure 9: Full import completed

The command imports three node files and three relationship files with `--id-type=integer`. The summary reports 704,600 nodes, 20,536,900 relationships and 62,275,900 properties, imported in 1 min 49 s with 1.04 GiB peak memory.

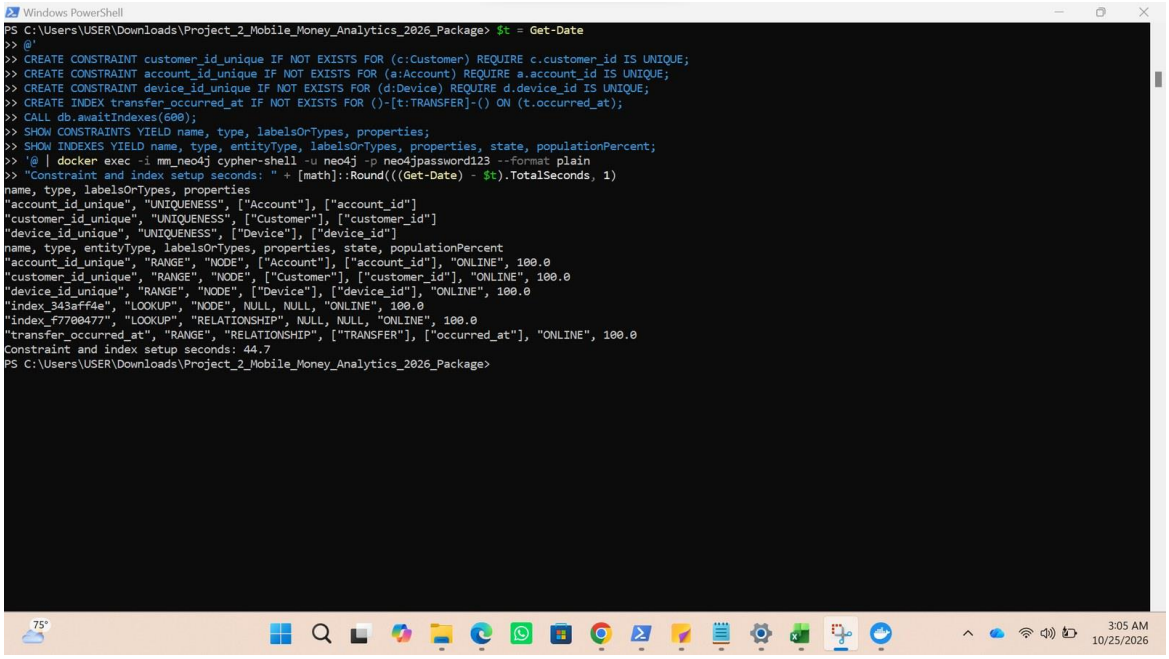


Figure 10: Constraints and indexes online

Three uniqueness constraints and the `TRANSFER.occurred_at` range index are created, and every index is ONLINE at 100%. Setup took 44.7 s.

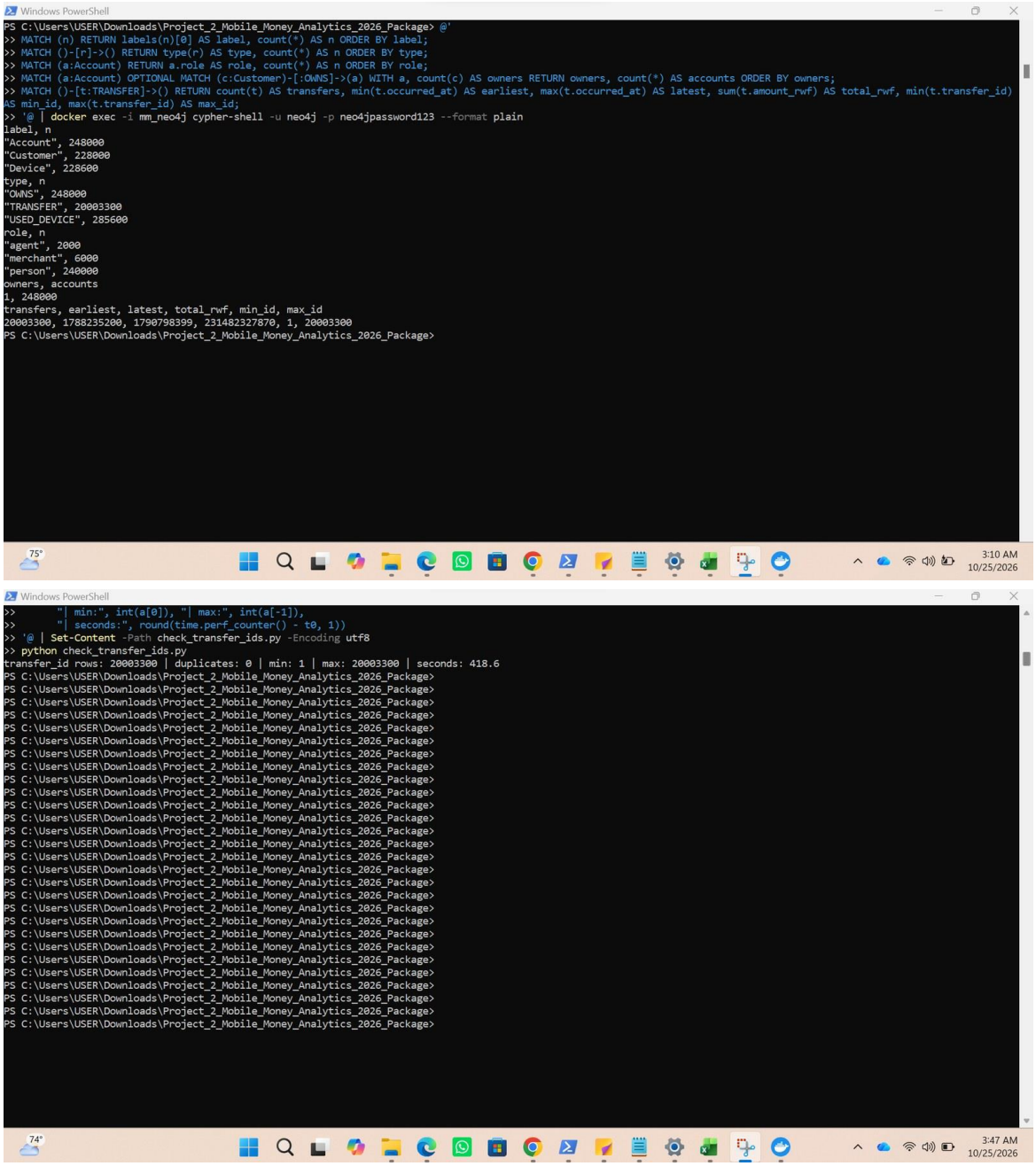


Figure 11: Node, relationship, role, owner, transfer_id and audit checks

Node counts per label, relationship counts per type, account roles, exactly one owner for each of the 248,000 accounts, and the transfer audit all match the capture. `check_transfer_ids.py` reads all 20,003,300 transfer_id values and finds 0 duplicates.

3.3 Inspect One Account in Both Models

Account 998 traced in both models (Figure 12): role person; owner Customer 998 (Kigali); devices 998 and 228001 (handsets, days 0–29, 39 and 16 observations); first outgoing transfer 113540 to account 240004 at 1788245009 (1 September, 08:43:29 CAT) for 2,100 RWF. Both queries return the same rows. SQL needs four JOINs on foreign keys; Cypher matches one pattern, `(c)-[:OWNS]->(a)-[:USED_DEVICE]->(d)` with `(a)-[t:TRANSFER]->(r)`.

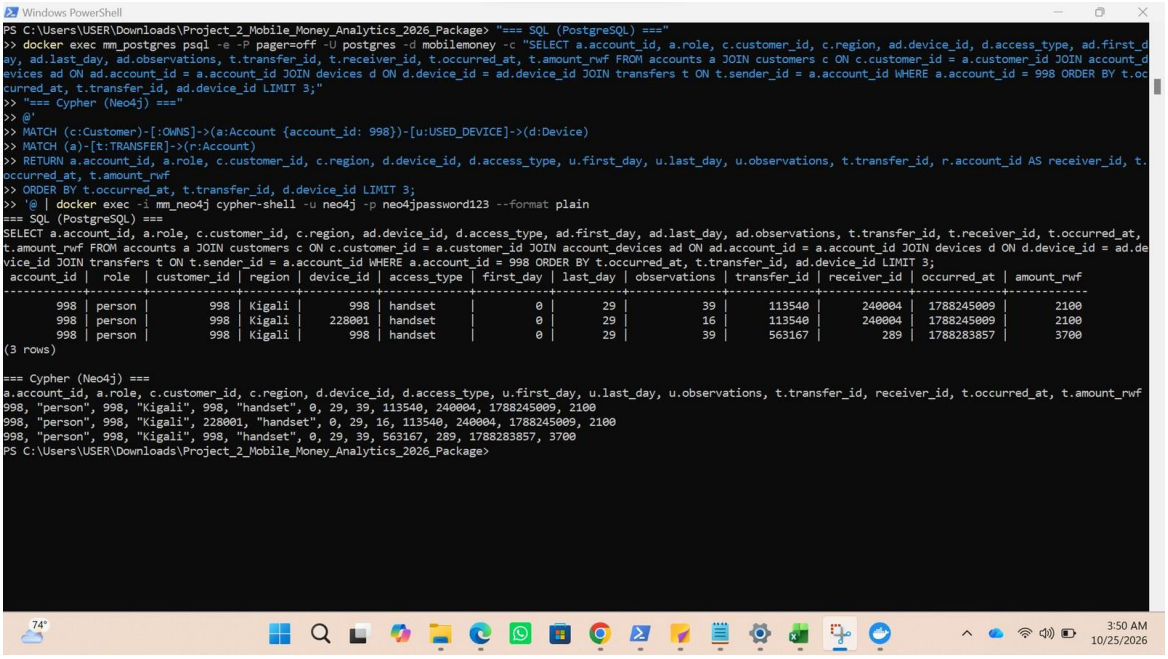


Figure 12: Account 998 traced with SQL JOINs (top) and Cypher MATCH (bottom)

Both queries return the same three rows: account 998 (person, customer 998, Kigali) with devices 998 and 228001, and its first outgoing transfers 113540 and 563167.

From foreign keys to relationships. A foreign key value such as `accounts.customer_id` is resolved at query time by an index lookup. In the graph the same fact is stored as an OWNS relationship attached to both nodes, so following it needs no lookup.

If repeated payments were collapsed into one edge per account pair, each payment's `transfer_id`, `occurred_at` and `amount_rwf` would be lost, along with the count and order of the payments. Q1 hourly totals, Q2 daily totals, and the time and amount rules of Q5 and Q6 could no longer be answered.

4 Answer Routine Business Questions

4.1 Q1 Account Activity

Hour boundaries are converted from CAT to epoch seconds in Python (`hour_bounds` in `common.py`), so both engines receive the same integer parameters. Both queries return the outgoing count and total RWF in the hour for all receiver roles, with zeros when nothing matches.

Set	Account	Outgoing count	Total RWF	SQL = Cypher
A	998	1	200,000	True
B	147228	1	1,050,000	True
C	206978	1	300,000	True
D	237998	1	45,000	True
E	144228	1	40,000	True
F	203978	1	30,000	True

Table 8: Q1 results, 09:45–10:45 CAT

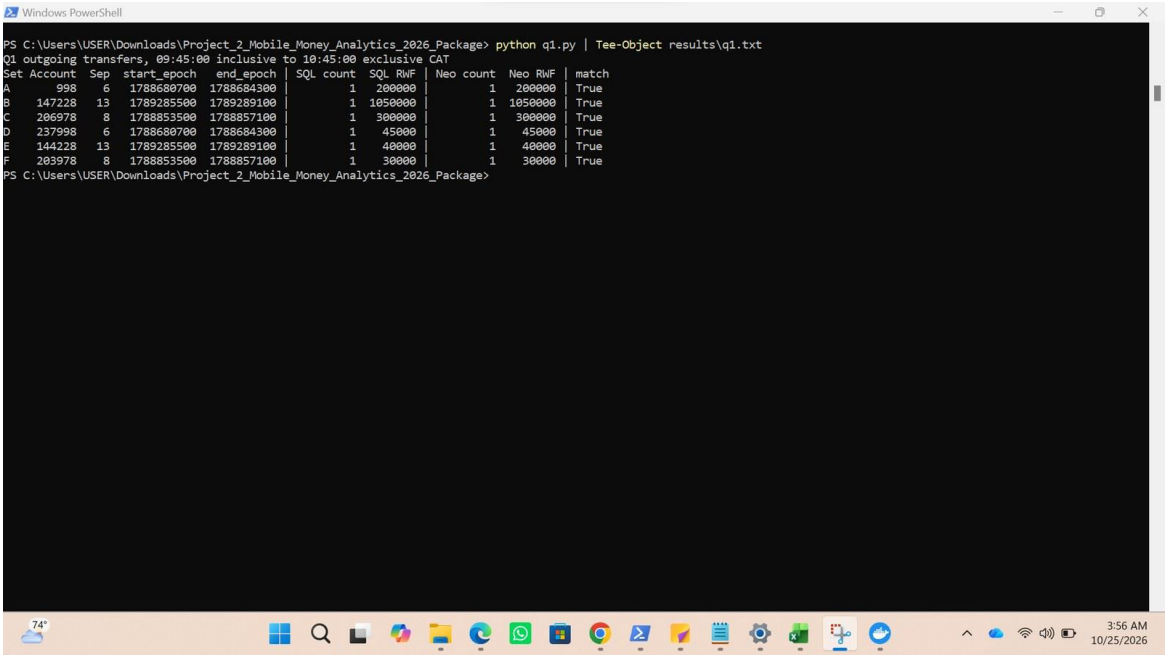


Figure 13: Q1: all six paired results

For each set the SQL and Cypher count and total are identical: each review account sent one transfer in its hour, from 30,000 RWF (F) to 1,050,000 RWF (B).

Outgoing value is not a balance. It ignores money received, the opening balance, cash-in and cash-out at agents, fees and reversals. An account can send a large amount in an hour while its balance stays the same or grows.

4.2 Q2 Daily Activity

SQL groups by `(to_timestamp(occurred_at) AT TIME ZONE 'Africa/Kigali')::date`; Cypher by `date(datetime({epochSeconds: …, timezone: "Africa/Kigali"}))`. Both return all 30 days, and the daily counts and values add up exactly to the capture totals (20,003,300 transfers and 231,482,327,870 RWF).

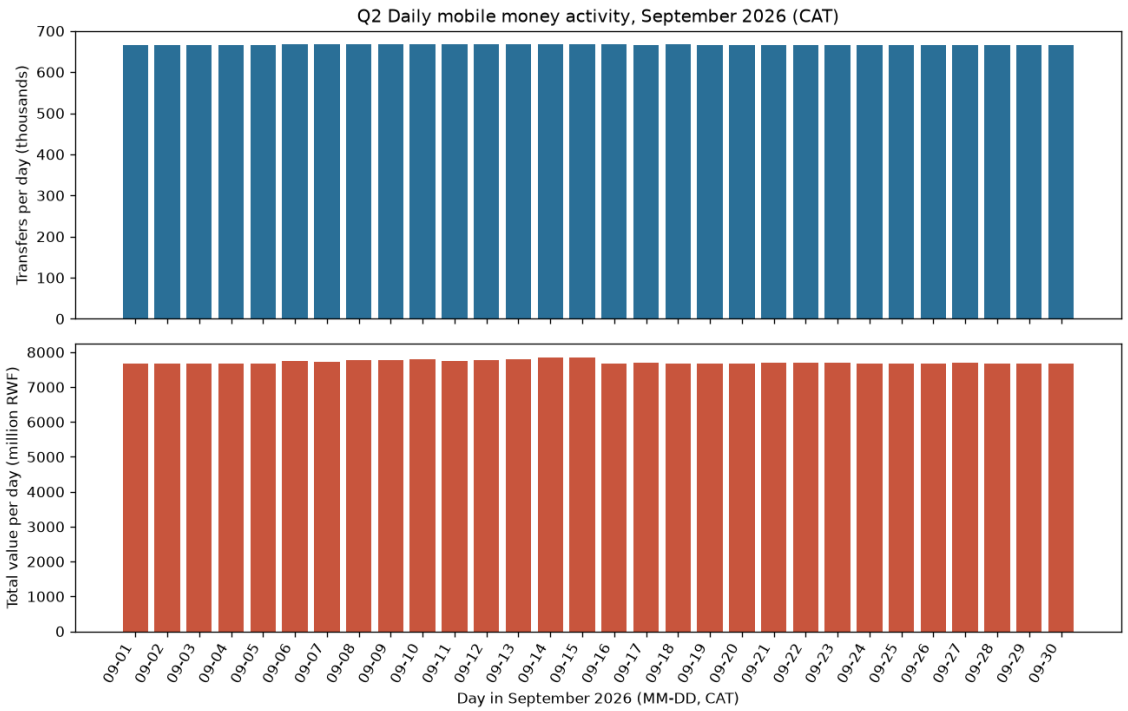
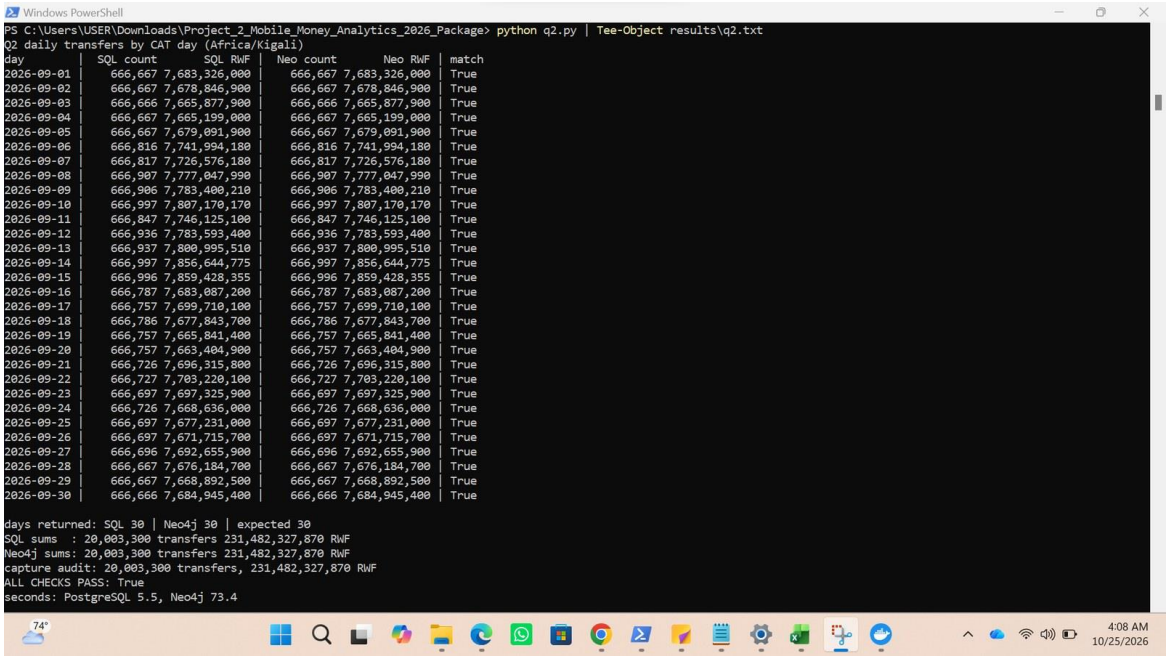


Figure 14: Daily transfer count (thousands) and value (million RWF), September 2026



```
PS C:\Users\USER\Downloads\Project_2_Mobile_Money_Analytics_2026_Package> python q2.py | Tee-Object results\q2.txt
Q2 daily transfers by CAT day (Africa/Kigali)
day        |   SQL count       SQL RWF |   Neo count       Neo RWF | match
2026-09-01 |     666,667 7,683,326,000 |     666,667 7,683,326,000 | True
2026-09-02 |     666,667 7,678,846,900 |     666,667 7,678,846,900 | True
2026-09-03 |     666,666 7,665,877,900 |     666,666 7,665,877,900 | True
2026-09-04 |     666,667 7,665,199,000 |     666,667 7,665,199,000 | True
2026-09-05 |     666,667 7,679,091,900 |     666,667 7,679,091,900 | True
2026-09-06 |     666,816 7,741,994,180 |     666,816 7,741,994,180 | True
2026-09-07 |     666,817 7,726,576,180 |     666,817 7,726,576,180 | True
2026-09-08 |     666,907 7,777,047,990 |     666,907 7,777,047,990 | True
2026-09-09 |     666,906 7,783,400,210 |     666,906 7,783,400,210 | True
2026-09-10 |     666,997 7,807,170,170 |     666,997 7,807,170,170 | True
2026-09-11 |     666,847 7,746,125,100 |     666,847 7,746,125,100 | True
2026-09-12 |     666,936 7,783,593,400 |     666,936 7,783,593,400 | True
2026-09-13 |     666,937 7,800,995,510 |     666,937 7,800,995,510 | True
2026-09-14 |     666,997 7,856,644,775 |     666,997 7,856,644,775 | True
2026-09-15 |     666,996 7,859,428,355 |     666,996 7,859,428,355 | True
2026-09-16 |     666,787 7,683,087,200 |     666,787 7,683,087,200 | True
2026-09-17 |     666,757 7,699,710,100 |     666,757 7,699,710,100 | True
2026-09-18 |     666,786 7,677,843,700 |     666,786 7,677,843,700 | True
2026-09-19 |     666,757 7,665,841,400 |     666,757 7,665,841,400 | True
2026-09-20 |     666,757 7,663,404,900 |     666,757 7,663,404,900 | True
2026-09-21 |     666,726 7,696,315,800 |     666,726 7,696,315,800 | True
2026-09-22 |     666,727 7,703,220,100 |     666,727 7,703,220,100 | True
2026-09-23 |     666,697 7,697,325,900 |     666,697 7,697,325,900 | True
2026-09-24 |     666,726 7,668,636,000 |     666,726 7,668,636,000 | True
2026-09-25 |     666,697 7,677,231,000 |     666,697 7,677,231,000 | True
2026-09-26 |     666,697 7,671,715,700 |     666,697 7,671,715,700 | True
2026-09-27 |     666,696 7,692,655,900 |     666,696 7,692,655,900 | True
2026-09-28 |     666,667 7,676,184,700 |     666,667 7,676,184,700 | True
2026-09-29 |     666,667 7,668,892,500 |     666,667 7,668,892,500 | True
2026-09-30 |     666,666 7,684,945,400 |     666,666 7,684,945,400 | True

days returned: SQL 30 | Neo4j 30 | expected 30
SQL sums  : 20,003,300 transfers 231,482,327,870 RWF
Neo4j sums: 20,003,300 transfers 231,482,327,870 RWF
capture audit: 20,003,300 transfers, 231,482,327,870 RWF
ALL CHECKS PASS: True
seconds: PostgreSQL 5.5, Neo4j 73.4
```

Figure 15: Q2: 30 days in both engines and reconciliation to capture totals

All 30 CAT days match between the engines, and the sums reconcile to 20,003,300 transfers and 231,482,327,870 RWF, equal to the capture audit (ALL CHECKS PASS: True). This run took 5.5 s in PostgreSQL and 73.4 s in Neo4j.

Pattern. The number of transfers is almost constant at about 667,000 per day, but daily value rises slightly from 6 to 15 September (about 7.75–7.85 billion RWF against about 7.68 billion on other days). More money moved per transfer in mid-month, not more transfers.

Value is not revenue. Transfer value is customers' money. The provider earns fees, which depend on the tariff, the transfer type and who pays; many transfers carry no fee, and cash-out fees are charged separately. Revenue would need fee records.

4.3 Q3 Shared Devices

SQL self-joins `account_devices` on `device_id`; Cypher matches `(a)-[:USED_DEVICE]->(d)<-[:USED_DEVICE]-(b)`. Both keep pairs where `max(first_day) <= min(last_day)`, exclude the review account and include both device types.

Set	Account	Other accounts	Access types	Match
A	998	4	handset 4	True
B	147228	9	handset 9	True
C	206978	9	handset 9	True
D	237998	4	handset 4	True
E	144228	9	handset 9	True
F	203978	9	handset 9	True

Table 9: Q3 overlapping device sharing (44 rows in total)

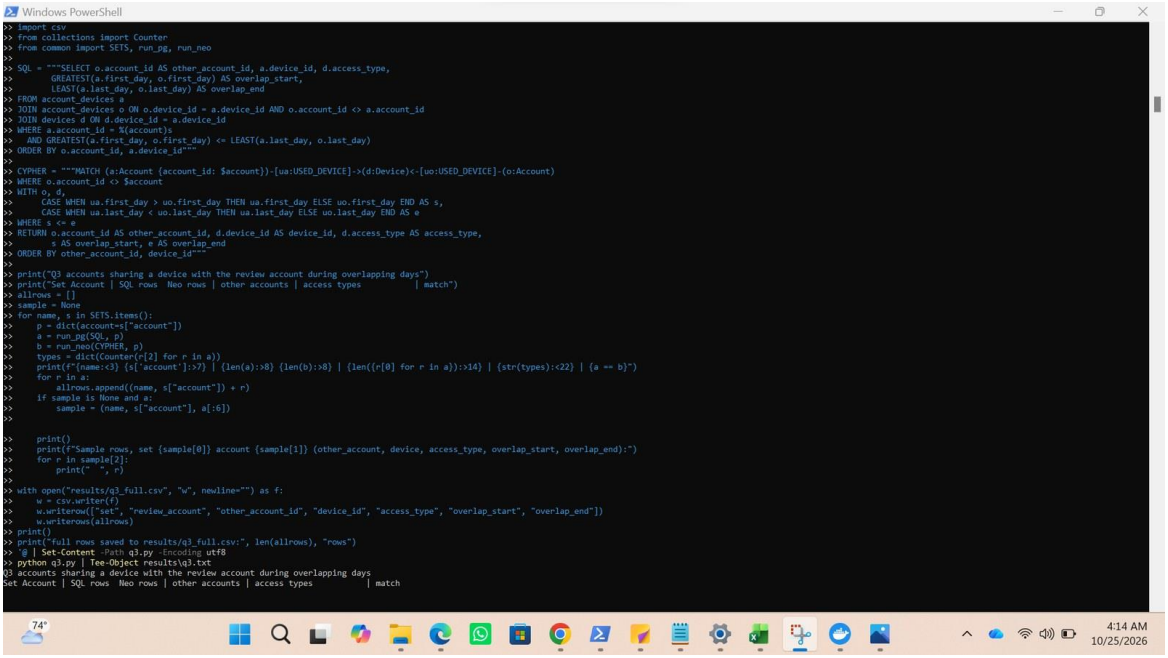


Figure 16: Q3: matching results and counts for all six sets

Both engines return the same rows for every set (4 or 9 other accounts, all on handsets). The query keeps only pairs whose larger first_day is not after the smaller last_day.

Legitimate sharing. Account 998 shares device 998 with account 220998, which belongs to the same customer (998). One person running two wallets on one phone is ordinary, as are family members sharing a handset or customers served on an agent's phone.

Overlap is not simultaneous use. Intervals [2,7] and [5,11] overlap on days 5–7, but the two accounts may have used the device at different hours, or on different days within that range.

5 Investigate Connected Accounts

5.1 Q4 Ownership and Device Network

SQL builds an `edges` CTE from ownership and device rows in both directions, carrying the node kind with the ID so that Customer 7 and Account 7 differ, and walks it with `WITH RECURSIVE`; `UNION` removes repeated (kind, id, depth) states. Cypher uses a bounded undirected path `(a)-[:OWNS|USED_DEVICE*1..N]-(b:Account)`, then `DISTINCT` accounts before grouping by region.

Set	Account	Depth 4	Depth 6	Depth 8	Depth 10
A	998	10	43	175	187
B	147228	46	178	188	228
C	206978	146	483	832	1,433
D	237998	10	44	180	192
E	144228	47	183	193	230
F	203978	147	488	837	1,426

Table 10: Q4 distinct accounts reached; all 24 comparisons match

Prediction (recorded before testing depths 8 and 10). From depth 4 to 6 every set grew 3.3 to 4.4 times, so we expected about 3.5 times growth to depth 8 and 3 times to depth 10: depth 8 A 150, B 620, C 1,700, D 150, E 640, F 1,700; depth 10 A 450, B 1,900, C 5,000, D 450, E 1,900, F 5,100. The actual counts were far lower (for example B 188 and 228, C 832 and 1,433) because each network saturated: most accounts reachable through ownership and devices were already found by depth 6 to 8, so growth slowed instead of continuing.

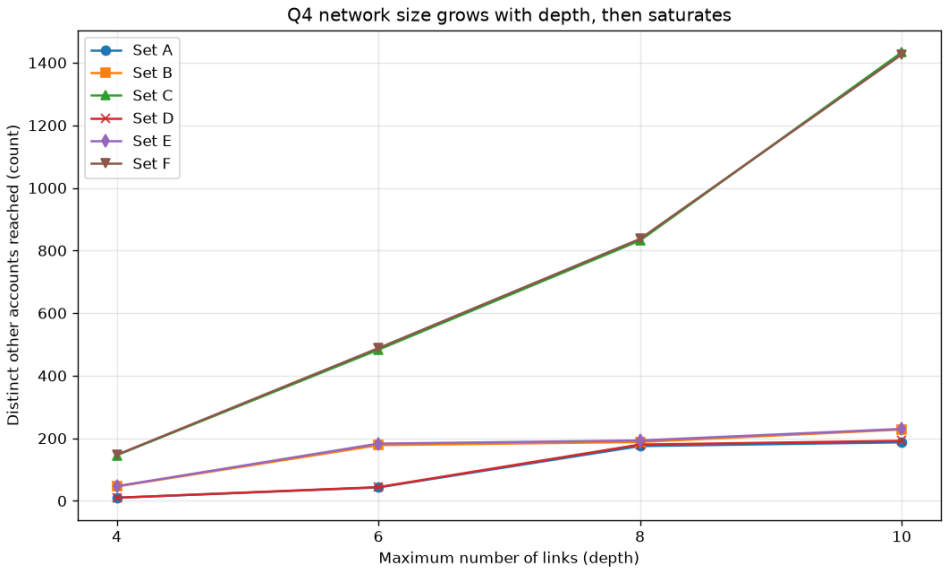


Figure 17: Q4 accounts reached against depth

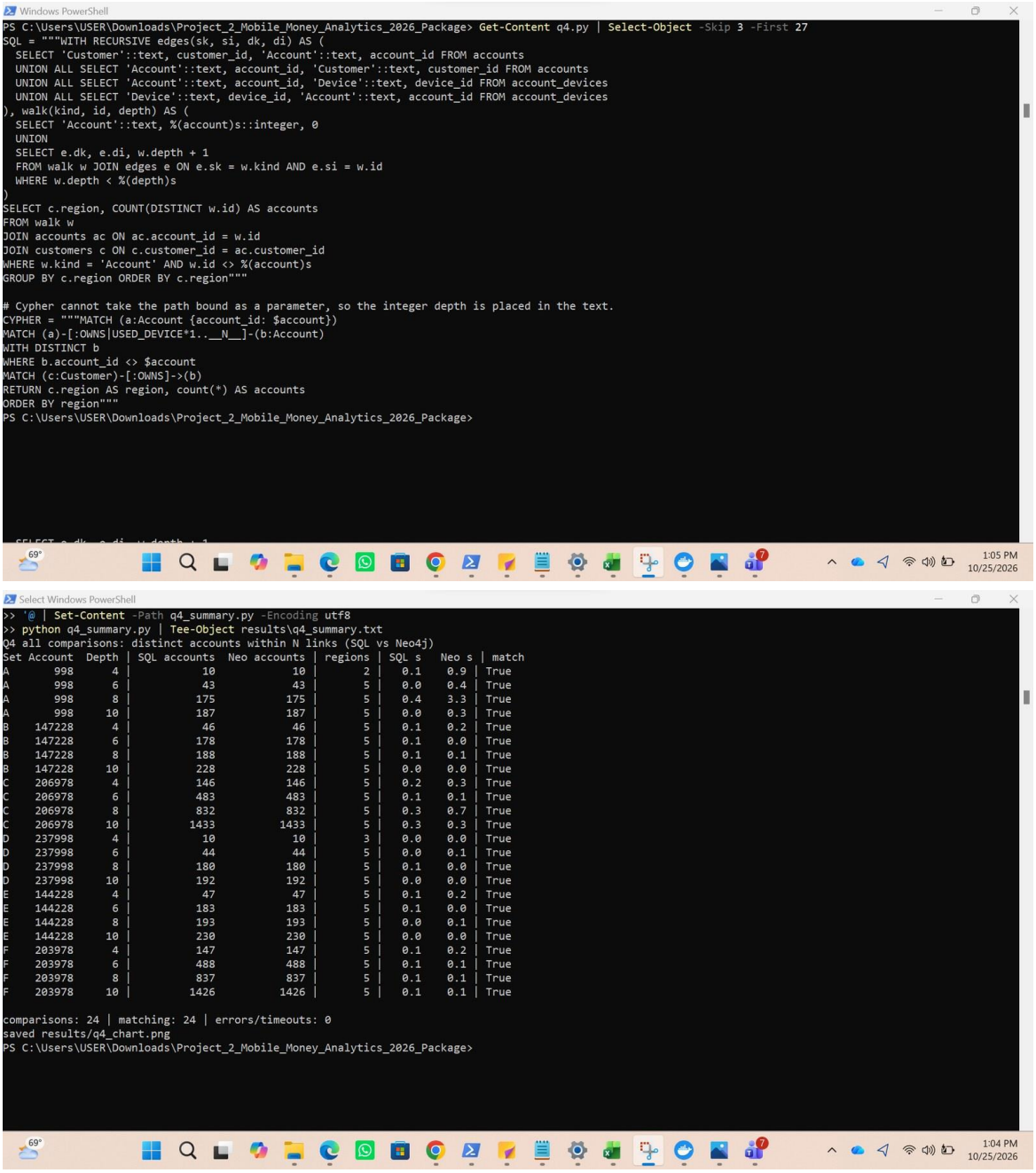


Figure 18: Q4: query pair and all 24 comparisons

The top screenshot shows the recursive SQL (edges CTE carrying node kind and ID) and the bounded Cypher pattern; the bottom shows all 24 set and depth counts matching, with 0 errors or timeouts.

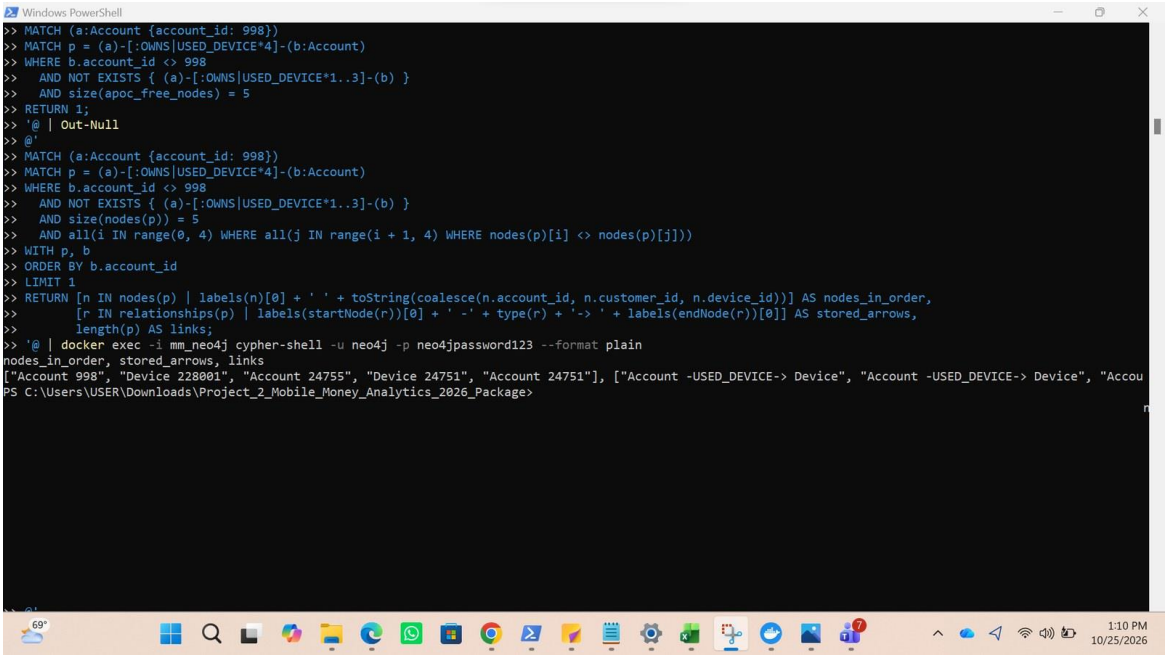


Figure 19: Q4: one supporting path

A four-link path from account 998 to account 24751 through device 228001, account 24755 and device 24751, listed with the stored arrow directions. It shows how an indirect association arises without any transfer.

Weak associations. At depth 4 the accounts reached share a customer or device with the review account or with its direct neighbours. By depth 10, set C reaches 1,433 accounts through long chains of shared phones and multi-wallet customers; a single public terminal or agent handset can join hundreds of unrelated people. Being reachable is an association, not evidence of a relationship.

5.2 Q5 Rapid Onward Payments

SQL joins three transfer aliases receiver-to-sender; Cypher matches three directed TRANSFER relationships. Both apply: four distinct accounts; the first transfer from the review account inside its hour; each later transfer strictly later and at most 600 s after the previous one; and `90*previous <= 100*next <= 105*previous` (BIGINT arithmetic in SQL).

Set	Account	Chains	Transfer IDs → final receiver
A	998	1	20000001, 20000002, 20000003 → 24755
B	147228	1	20000092, 20000093, 20000094 → 170985
C	206978	1	20000229, 20000230, 20000231 → 230735
D, E, F		0	–

Table 11: Q5 chains; SQL and Cypher match for all six sets

Transfer	From → To	Time (CAT, 6 Sep)	RWF	Gap	% of previous
20000001	998 → 8917	10:00:00	200,000	–	–
20000002	8917 → 16836	10:00:40	196,000	+40 s	98.0
20000003	16836 → 24755	10:01:20	192,080	+40 s	98.0

Table 12: Example chain, set A

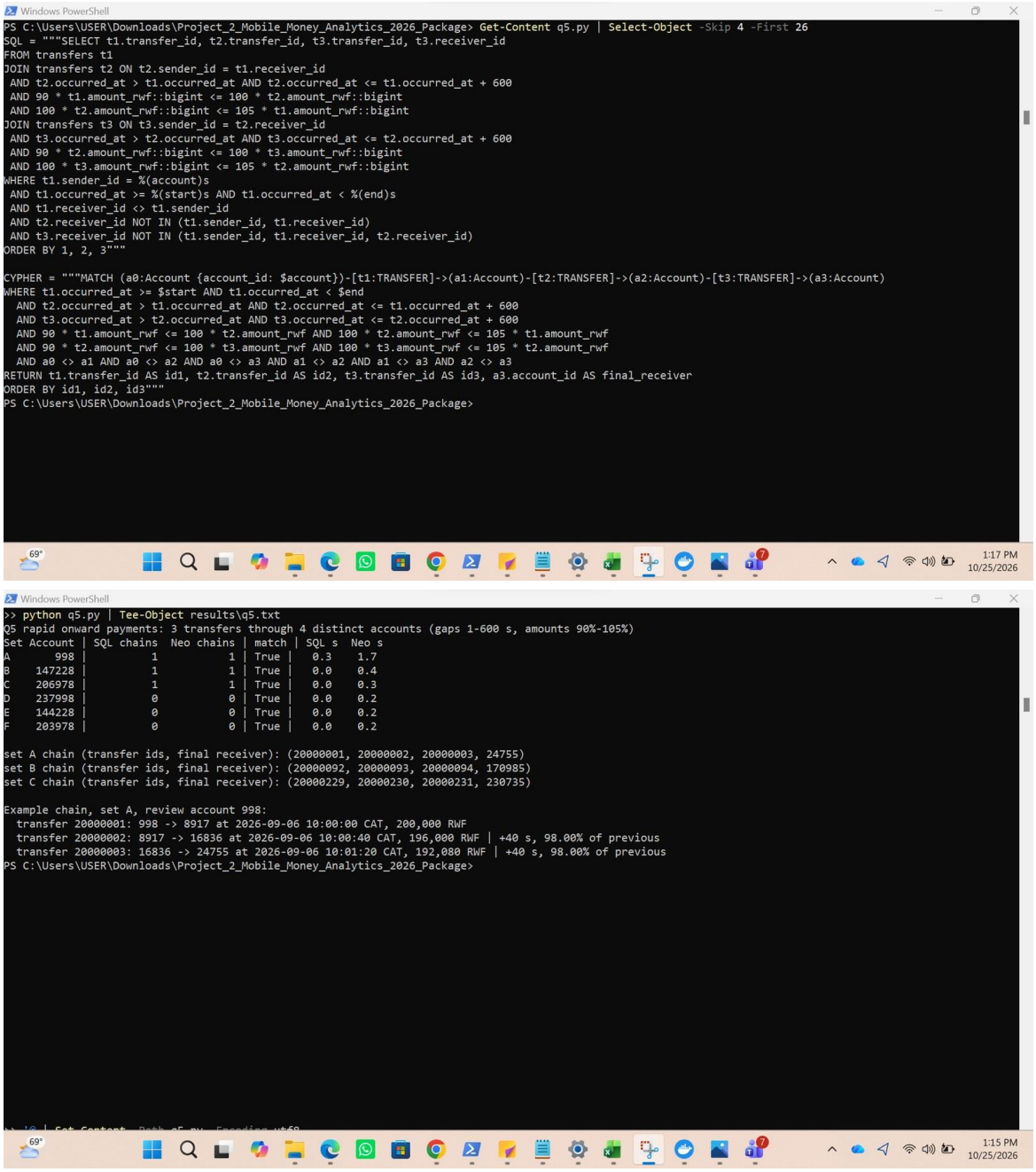


Figure 20: Q5: query pair, six matching counts and the example chain

The SQL joins three transfer aliases and the Cypher matches three relationships with the same time and amount rules. Both find one chain each for A, B and C and none for D, E and F, and the set A chain is printed with CAT times and amounts.

Not proof that the same funds moved. Money in a wallet is interchangeable: 8917 may have paid 16836 from an existing balance, and similar amounts can occur by coincidence. The records link events by account and time, not by funds.

5.3 Q6 Circular Payments

Recursive SQL carries the current account, the ordered transfer IDs, the first and last time and the last amount. It extends only eligible transfers (strictly later, inside the hour, total span at most

1,800 s, Q5 amount rule) and stops at the first return. Cypher uses a bounded path of 3–8 transfers with equivalent checks.

Set	Account	Cycles	Transfer-ID sequence (length)
A	998	1	20000001–20000004 (4)
C	206978	1	20000229–20000234 (6)
B, D, E, F		0	–

Table 13: Q6 cycles; SQL and Cypher match for all six sets

The set A cycle adds transfer 20000004: 24755 → 998 at 10:02:00 for 188,238 RWF (98% of the previous), closing the loop in 120 s (see the cycle diagram in Section 8.1).

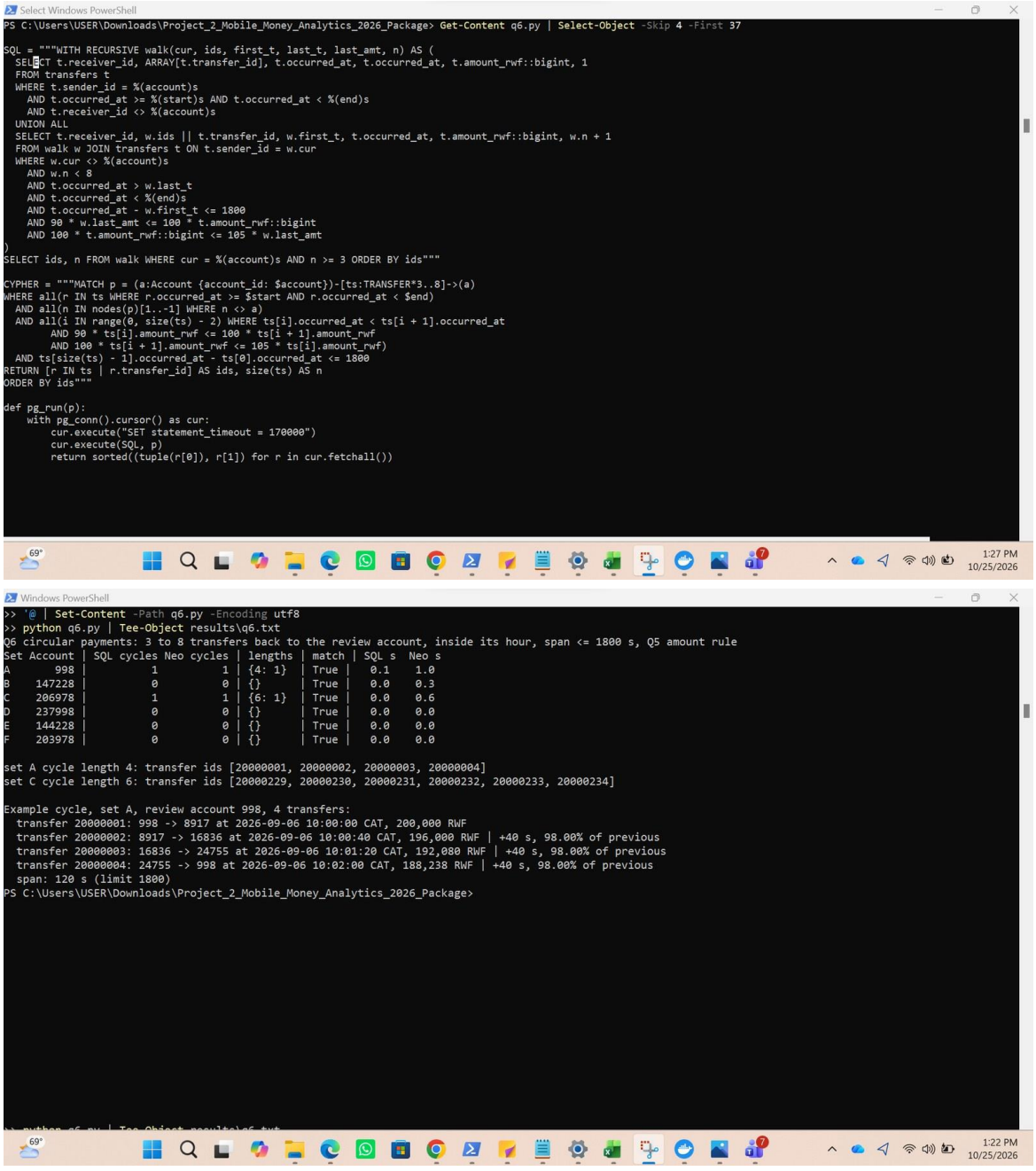


Figure 21: Q6: query pair, six matching counts and one cycle

The recursive SQL carries the current account, transfer IDs, first and last time and last amount. Both engines find a 4-transfer cycle for A and a 6-transfer cycle for C; the set A cycle closes in 120 s.

Further evidence requested: which device initiated each transfer, the KYC identity and relationships of the account owners, balances before and after, cash-out events after the cycle, and the accounts' history outside September.

5.4 Q7 Shortest Transfer Connection

SQL adapts the recursive approach to directed transfer neighbours, with every account on the route required to have role person during the search. Neo4j uses native shortest-path search, `SHORTEST 1 (a)(()-[:TRANSFER]->(:Account {role:'person'})){1,N}(b)`, which applies the restriction inside the search, not after choosing a path.

Set	Account	Destination	Bound 4	Bound 6	Match
A	998	24755	3	3	True
B	147228	186823	none	5	True
C	206978	6573	none	5	True
D	237998	21755	3	3	True
E	144228	183823	none	5	True
F	203978	3573	none	5	True

Table 14: Q7 minimum lengths; all 12 match, and all 6 retrieved paths are valid

For each successful case one path was retrieved separately and checked for IDs, direction, person roles, event existence and length (6/6 valid).

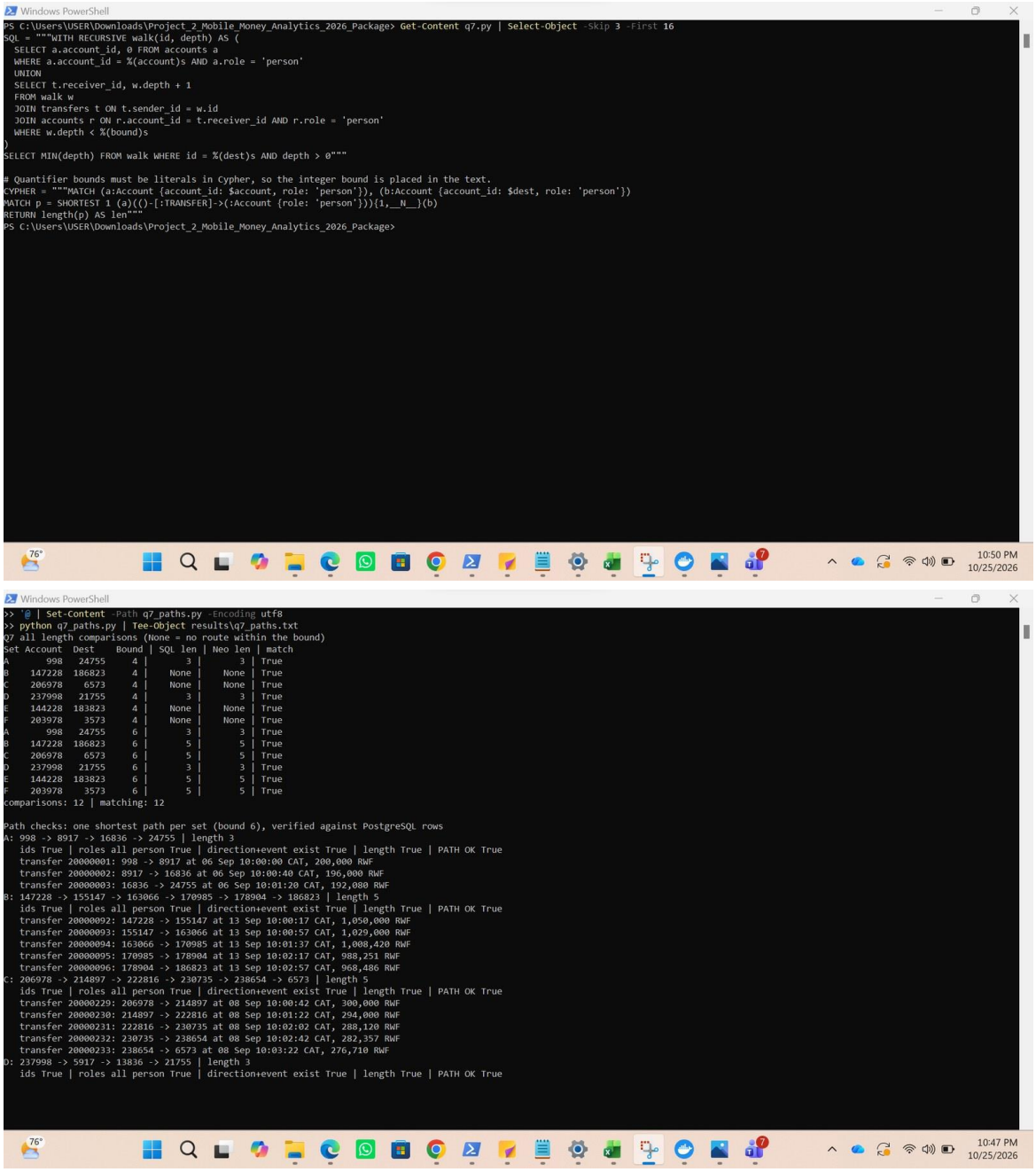


Figure 22: Q7: query pair, all 12 length comparisons and the six path checks

The query pair is followed by the 12 length comparisons (all match) and the path checks: one shortest path per successful case, verified for IDs, person roles, direction, event existence and length (all PATH OK).

No route within four transfers does not mean no connection. B, C, E and F have routes of length 5. A longer bound, routes through merchant or agent accounts (excluded here), other months or cash outside the system could still connect them.

A structural route need not show onward movement of funds. Q7 ignores time order and amounts, so a route can run backwards in time, as in the teaching example.

6 Check Correctness and Performance

6.1 Build One Comparison Program

`run_pg(query, params)` and `run_neo(query, params)` return every row; integers are normalised and rows sorted before exact comparison. Q7 lengths are compared separately from the supporting paths.

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Total
Matching	6/6	1/1	6/6	24/24	6/6	6/6	12/12	61/61

Table 15: 61-case comparison

Independent checks in plain Python against the captured CSVs, with no database, agree with the SQL results: device overlaps 6/6, Q5 chains 6/6 and Q6 cycles 6/6 (the set A cycle was found independently).

All 61 cases matched on the final run, so no mismatch remained to resolve.

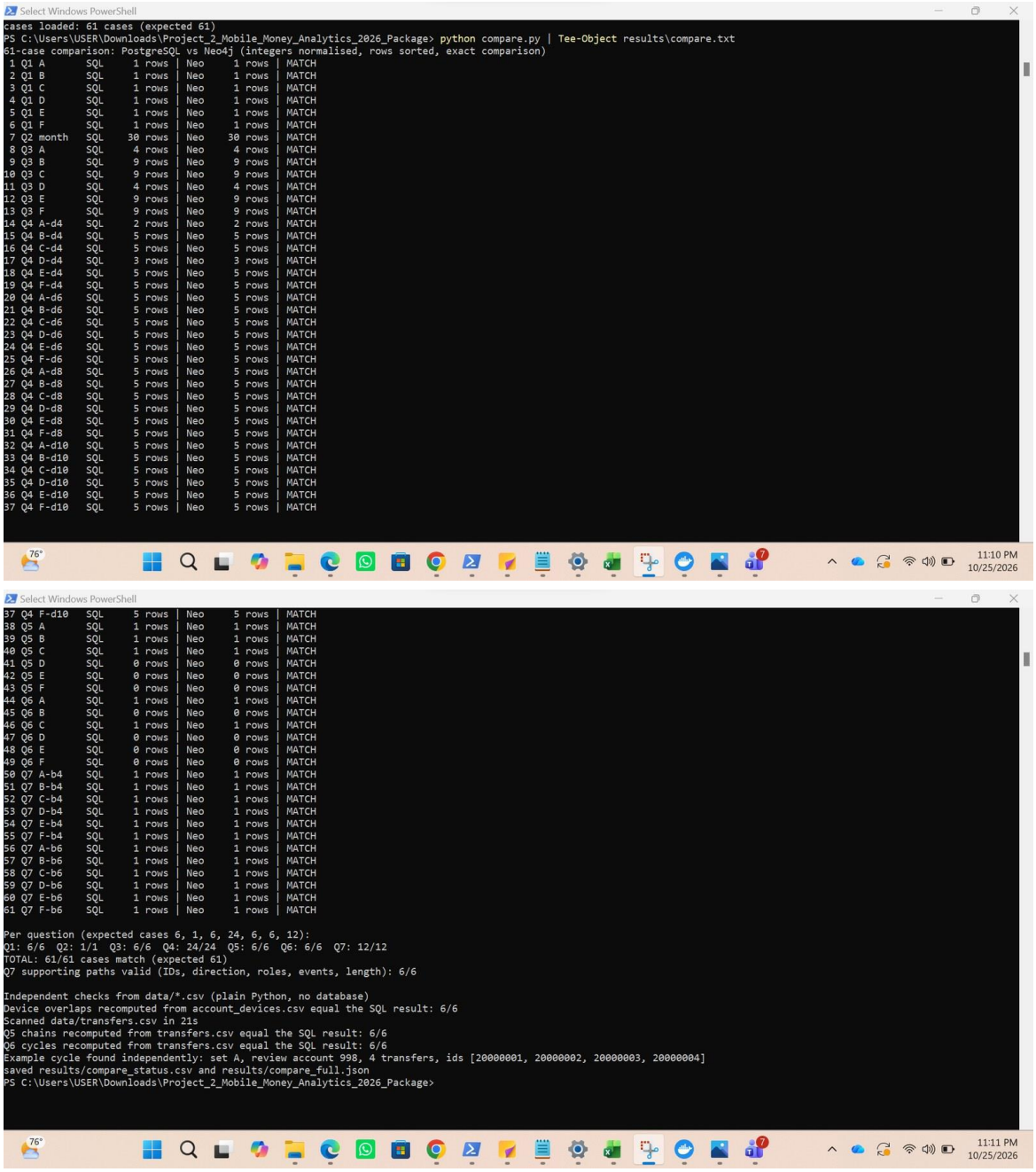


Figure 23: 61-case status table and independent checks

All 61 cases show MATCH (6, 1, 6, 24, 6, 6 and 12 per question). Q7 supporting paths are valid 6/6, and the independent Python checks on the CSVs agree for device overlaps, Q5 chains and Q6 cycles.

6.2 Time a Fixed Selection

Each engine was timed alone on the same machine with persistent connections and fixed settings: 21 cases, three untimed warm-ups and five measured runs each, timed with `perf_counter` from submission to the final row (180 s warm-up and 60 s measured limits). No case had errors or timeouts.

Case	PG median	PG min	PG max	Neo median	Neo min	Neo max	PG/Neo
Q1-B	1.5	1.2	1.7	8.5	8.0	12.4	0.18
Q1-E	1.6	1.3	1.7	8.3	7.0	9.9	0.19

Case	PG median	PG min	PG max	Neo median	Neo min	Neo max	PG/Neo
Q2-month	3,745.0	3,668.4	3,917.9	47,135.4	43,352.5	59,302.7	0.08
Q3-B	1.4	1.1	1.4	8.8	7.7	10.0	0.16
Q3-E	1.2	1.2	1.3	6.5	6.0	14.1	0.19
Q4-B-d4	1.8	1.8	2.4	7.3	6.7	8.4	0.25
Q4-E-d4	2.3	2.2	2.3	8.0	6.0	8.1	0.29
Q4-B-d6	3.6	3.4	5.1	8.7	7.3	9.1	0.41
Q4-E-d6	3.9	3.7	5.7	8.6	7.1	9.0	0.45
Q4-B-d8	6.5	6.1	7.3	8.6	8.0	9.3	0.76
Q4-E-d8	6.5	6.1	6.9	9.6	9.3	9.7	0.68
Q4-B-d10	8.6	7.6	9.6	7.1	6.3	7.8	1.21
Q4-E-d10	9.1	8.1	11.2	7.1	6.1	7.3	1.28
Q5-B	2.1	1.8	2.7	7.6	7.3	8.1	0.28
Q5-E	2.2	1.7	2.9	6.8	5.9	7.4	0.32
Q6-B	1.6	1.5	2.2	6.7	6.6	9.2	0.24
Q6-E	1.3	1.0	1.9	6.8	6.6	7.1	0.19
Q7-B-b4	264.6	222.2	274.7	12.8	12.6	13.5	20.7
Q7-E-b4	224.3	217.0	235.5	14.6	14.0	14.8	15.4
Q7-B-b6	19,205.2	18,287.9	23,603.5	16.0	14.2	17.7	1,200.3
Q7-E-b6	18,992.0	18,926.4	20,370.8	28.1	24.7	35.5	675.9

Table 16: Timing summary in milliseconds, 21 cases (original Q7 SQL); ratio = PostgreSQL median / Neo4j median

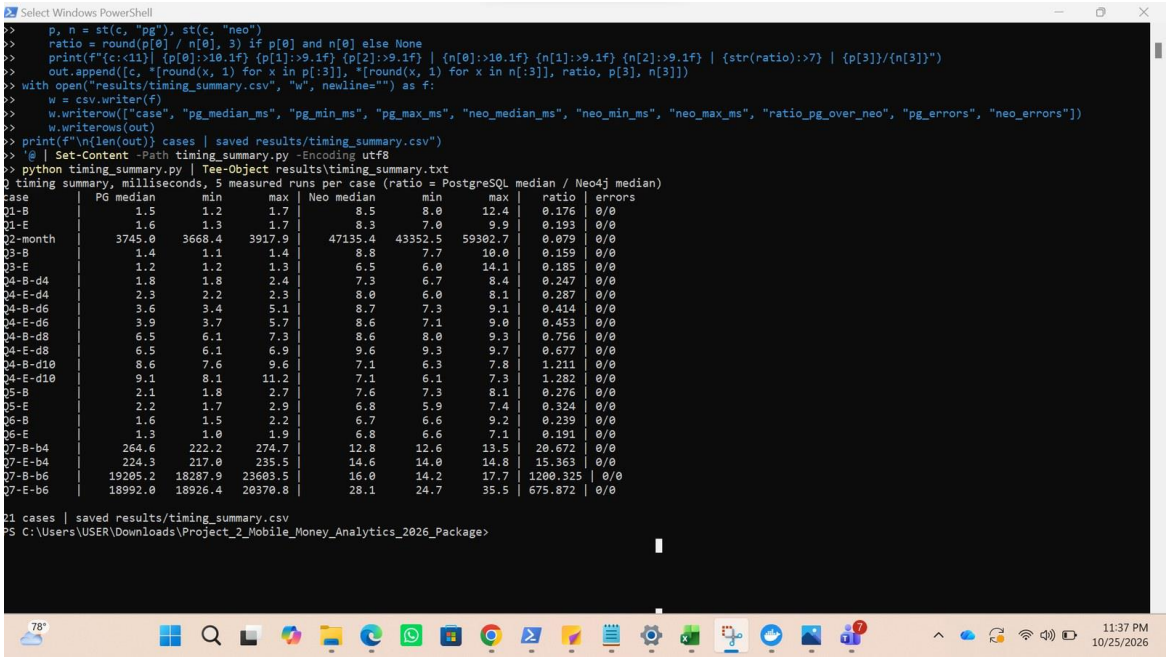


Figure 24: Timing summaries for both engines

Median, minimum and maximum milliseconds for all 21 cases in both engines, with the PostgreSQL/Neo4j median ratio and 0 errors or timeouts.

PostgreSQL wins the lookups and the monthly report; Neo4j has a floor of about 7 ms per query, which looks like fixed driver and session overhead and dominates small lookups. Neo4j wins Q4 at depth 10 and, by far, the original Q7.

Processing difference (Q7 B bound 6). Plans were collected outside timing (`results/plans/`).

Neo4j's ShortestPath operator finds both endpoints by unique index and runs one native search that stops at the destination: 19,481 DB hits. The recursive SQL expands every person account at every depth up to the bound: 427,955 (account, depth) states, 14.6 million edge probes and 60 million buffer hits (24.8 s under EXPLAIN ANALYZE). It reached the destination after about 0.9 s but kept going.

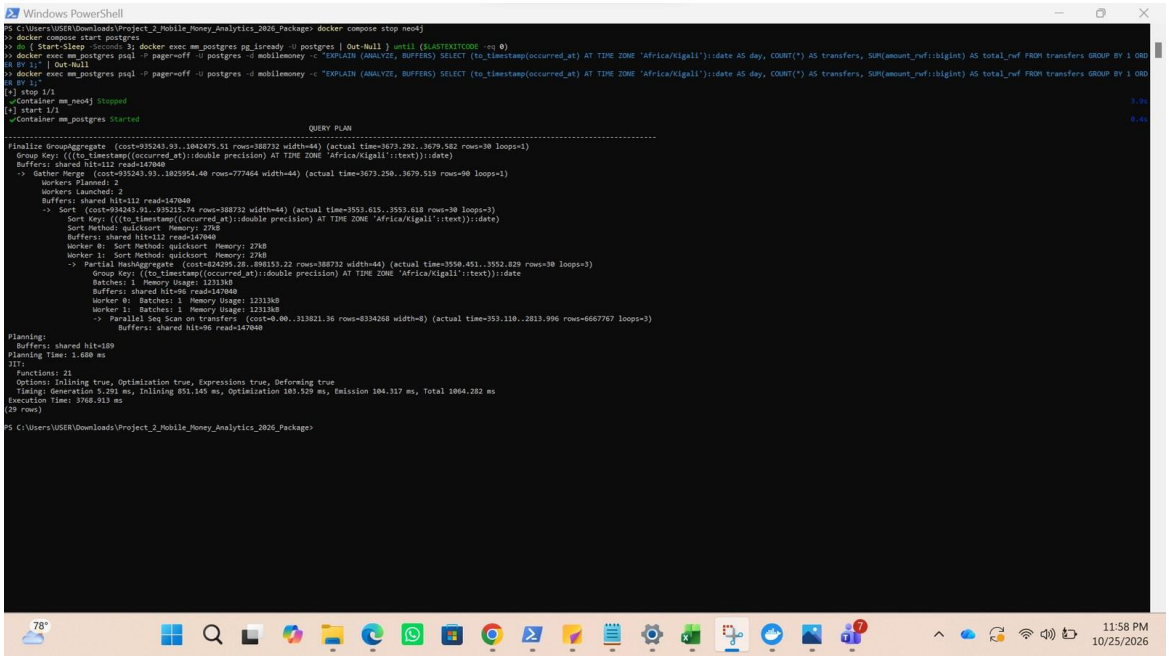


Figure 25: PostgreSQL plan, Q2

Parallel Seq Scan with two workers over 147,040 pages, partial and final aggregation into 30 days, and 3.77 s execution, of which 1.06 s is JIT compilation.

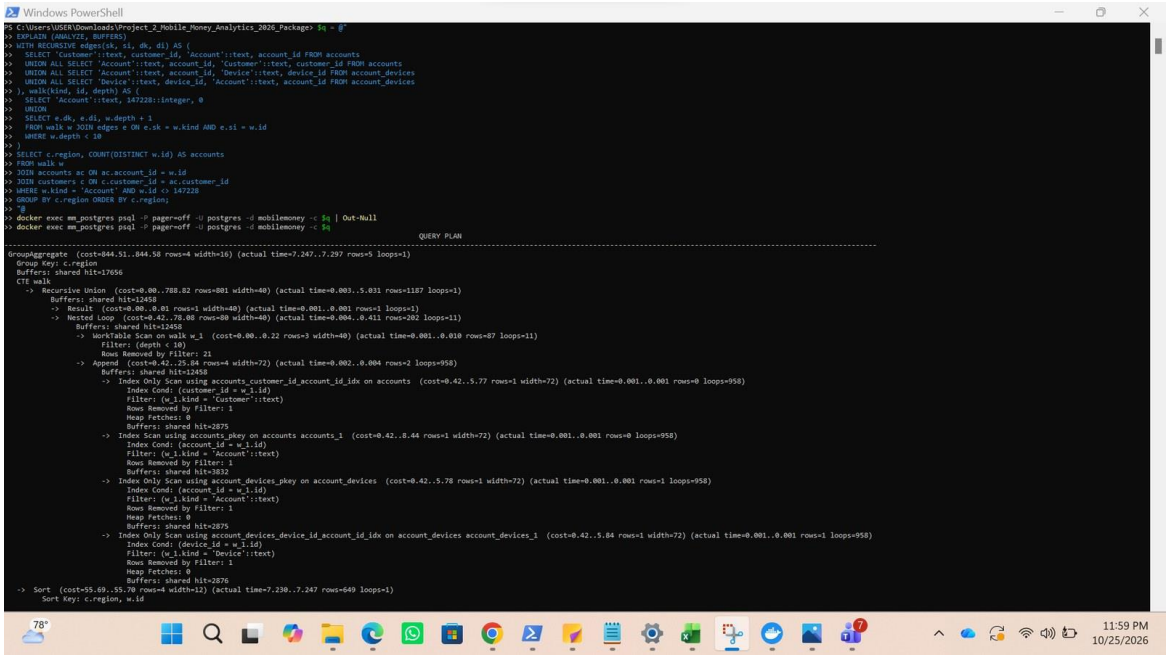


Figure 26: PostgreSQL plan, Q4 B depth 10 (top half)

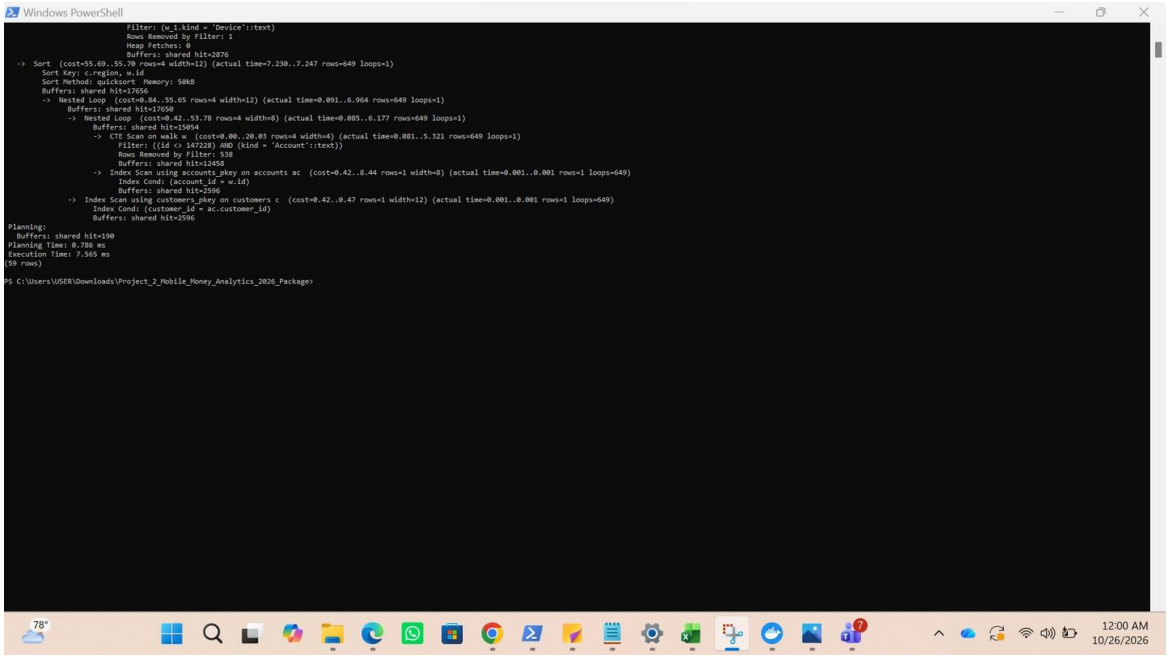


Figure 27: PostgreSQL plan, Q4 B depth 10 (bottom half)

Recursive Union over 11 rounds using index probes on the composite indexes (1,187 states), then the join to accounts and customers: 7.6 ms and 17,656 buffer hits.

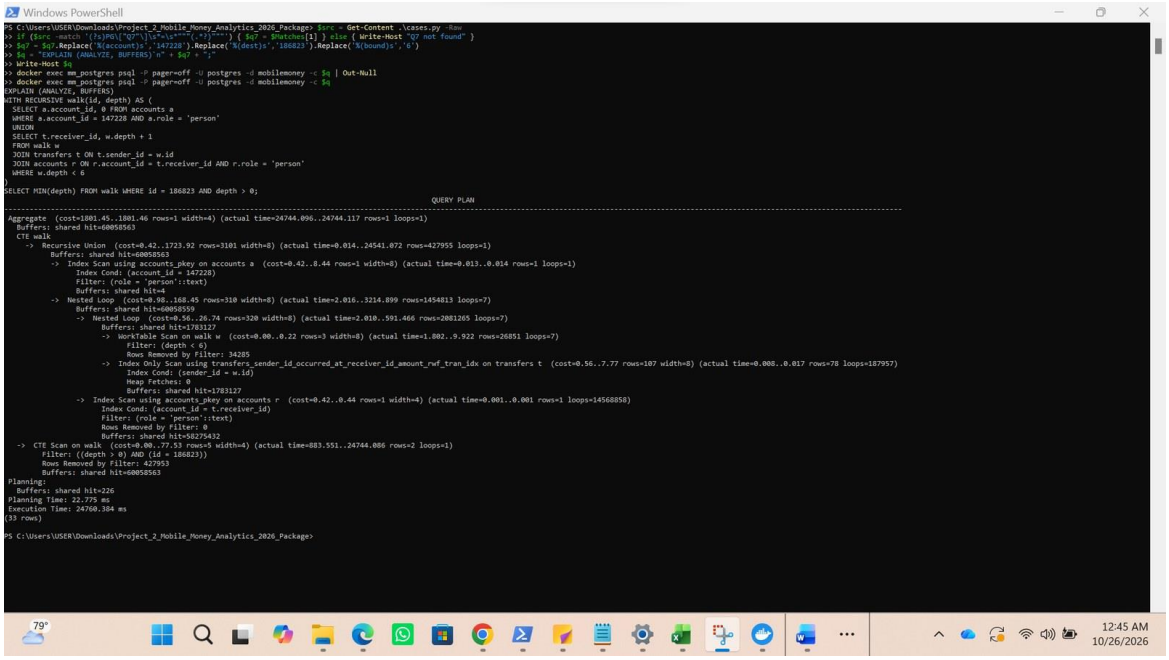


Figure 28: PostgreSQL plan, Q7 B bound 6, original SQL (the "before" plan)

Recursive Union of 7 rounds producing 427,955 (account, depth) states and 14.6 million edge probes (60 million buffer hits); 24.8 s.

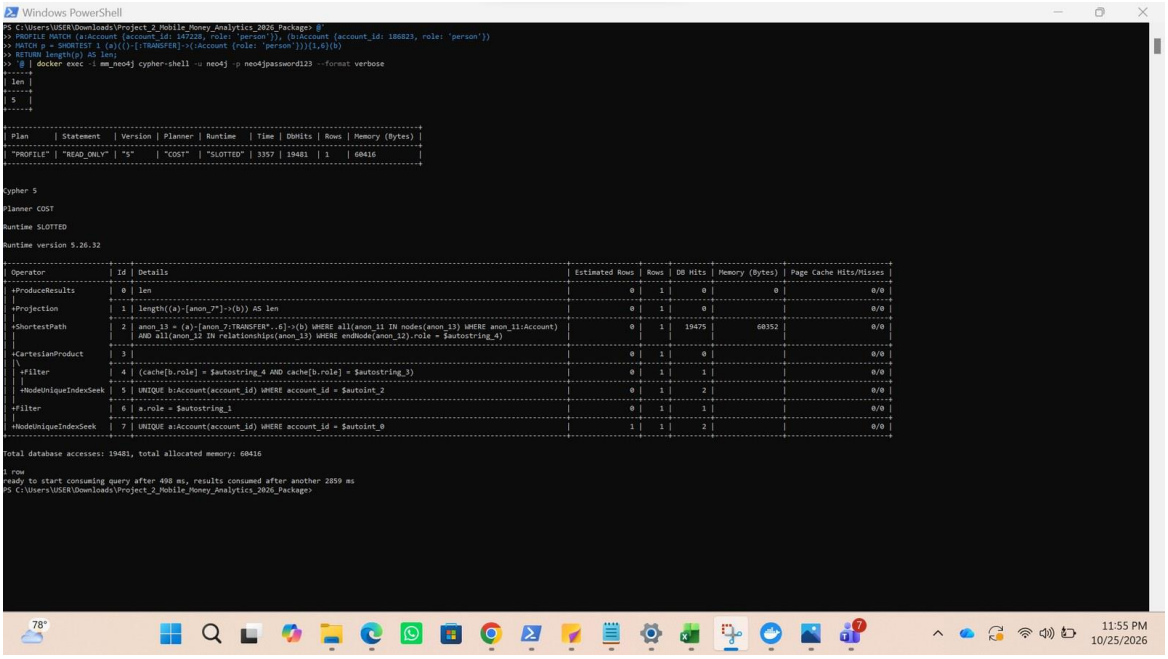


Figure 29: Neo4j PROFILE, Q7 B bound 6

One ShortestPath operator with the person filter inside the search, starting from both endpoints found by the unique index: 19,481 total DB hits.

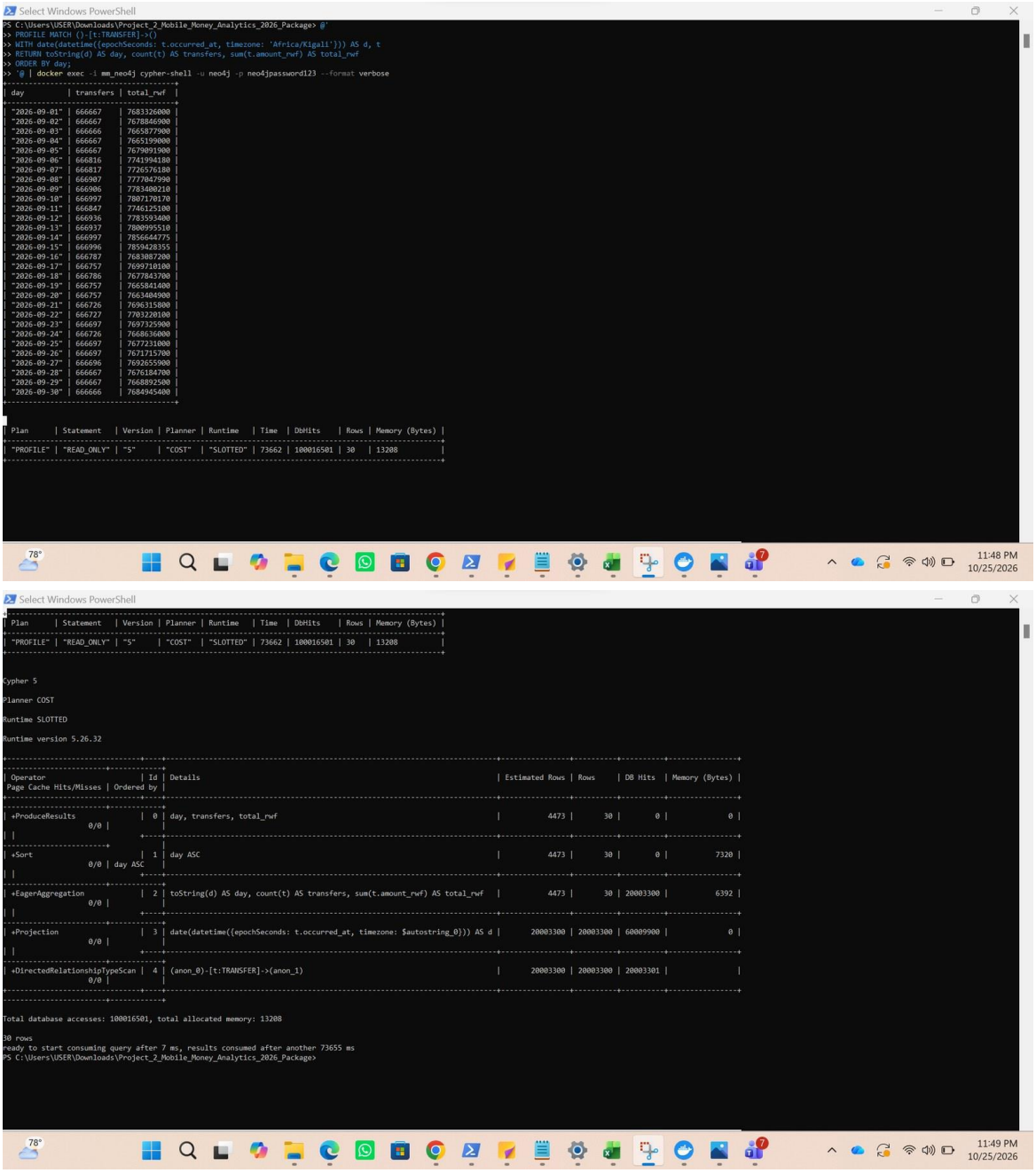


Figure 30: Neo4j PROFILE, Q2

A scan of every TRANSFER relationship with a date computed for each, then aggregation into 30 rows: 100,016,501 total DB hits.

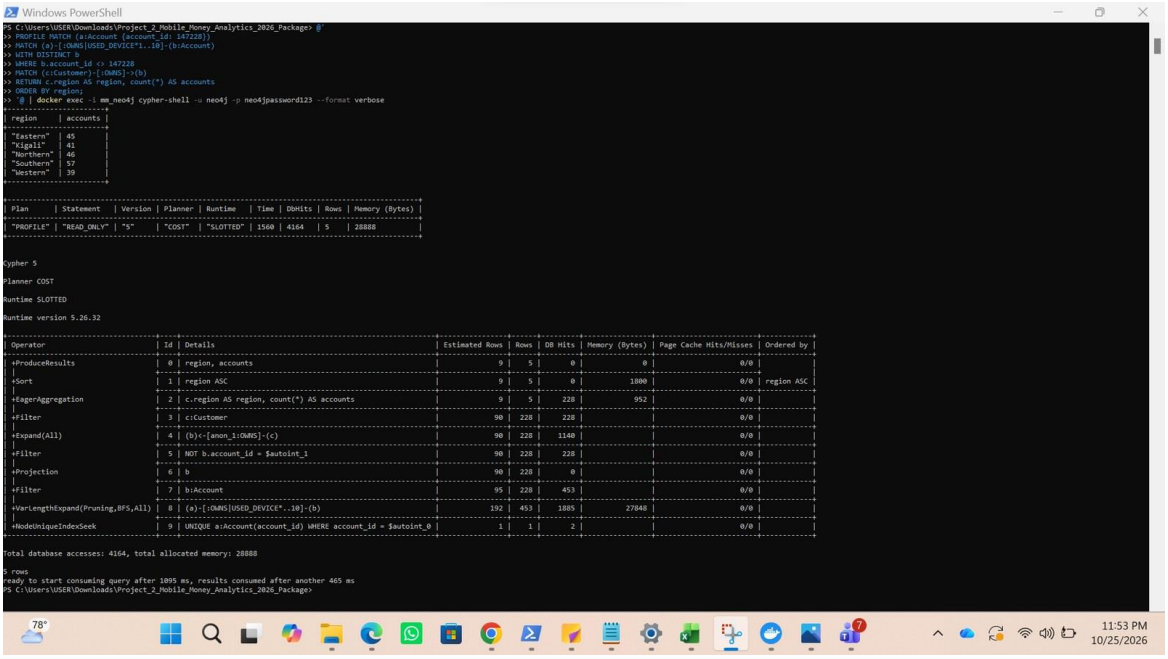


Figure 31: Neo4j PROFILE, Q4 B depth 10

Variable-length expansion over OWNS and USED_DEVICE, DISTINCT accounts, then grouping by region: 4,164 total DB hits.

6.3 Try One SQL Improvement

Wasted work. The plan shows that the original query stores (account, depth) pairs, so it re-expands the same accounts at every level, and it continues to depth 6 after reaching the destination. **Prediction:** a search that never expands depths 4–6 should be more than 50 times faster.

Change. `PG["Q7_v2"]` searches forward from the review account for ⌈b/2⌉ steps and backward from the destination for ⌊b/2⌋ steps (using the receiver index), then joins where they meet and takes the minimum. Any route of length L ≤ b has an account at position ⌈L/2⌉ that both searches reach, so the minimum is preserved. Both versions are kept. The "before" plan is Figure 28 in Section 6.2 and is not repeated here.

Case	Original (ms)	v2 median (ms)	Speed-up	Neo4j (ms)	v2 / Neo4j
B bound 4	264.6	8.9	29.7×	12.8	0.7
E bound 4	224.3	5.2	43.0×	14.6	0.4
B bound 6	19,205.2	37.9	506.1×	16.0	2.4
E bound 6	18,992.0	40.2	472.6×	28.1	1.4

Table 17: Revised SQL: 12/12 cases still correct; four revised timings

The v2 plan for B bound 6 expands 188 forward and 210 backward states, makes about 27,000 person checks and needs 111,730 buffer hits instead of 60 million (52 ms under EXPLAIN ANALYZE). **Outcome:** the prediction held. v2 is correct on all 12 cases, faster than Neo4j at bound 4 and only 1.4–2.4 times slower at bound 6. **v2 is the version used in Section 7 and in the final comparison** (B/E bound-6 medians 37.9 / 40.2 ms against Neo4j's 16.0 / 28.1 ms).

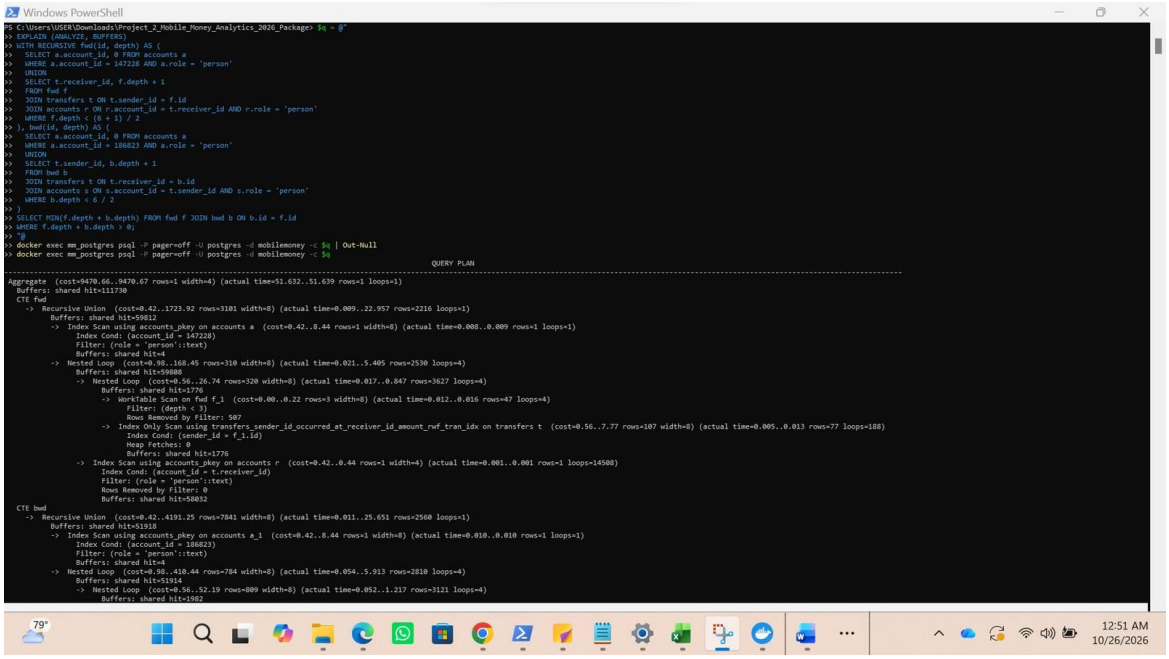


Figure 32: PostgreSQL plan, Q7 v2 (the "after" plan, top half)

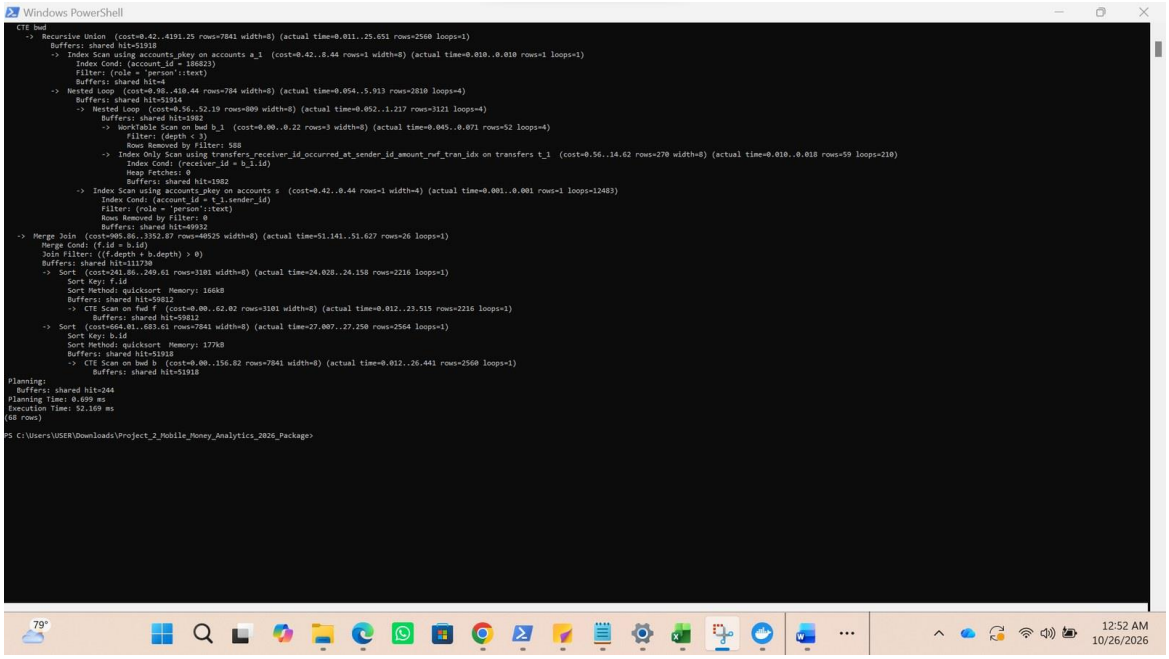


Figure 33: PostgreSQL plan, Q7 v2 (bottom half)

Two Recursive Unions, forward and backward to depth 3 (2,216 and 2,560 states), joined with a Merge Join on the meeting accounts: 111,730 buffer hits and 52 ms.

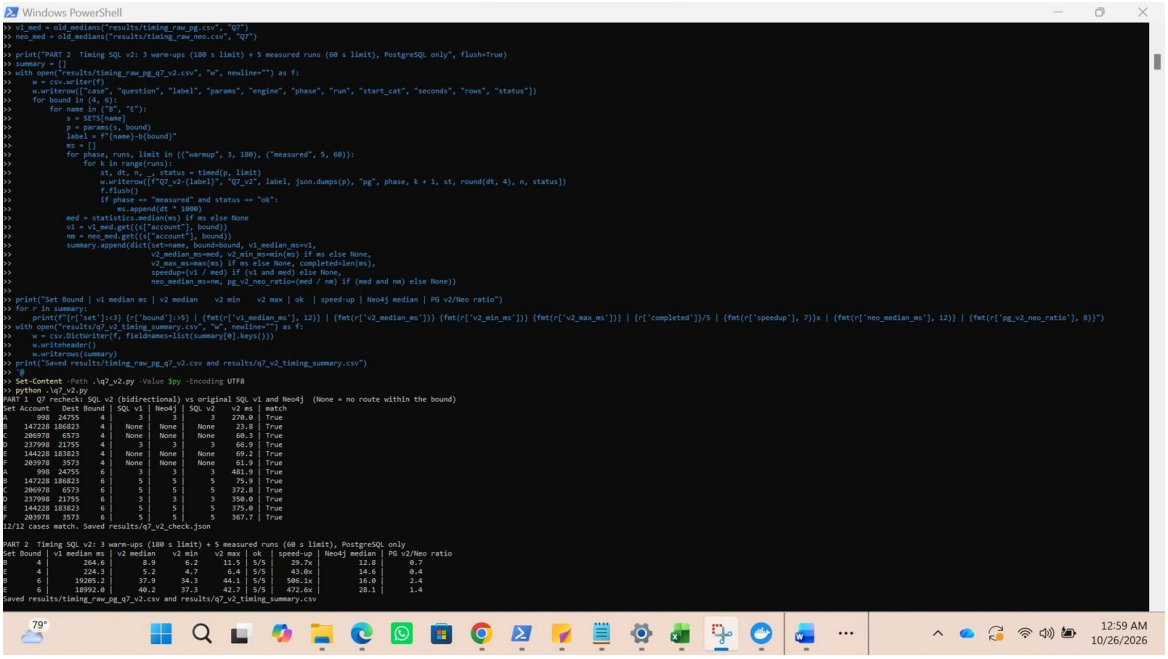


Figure 34: Q7 v2: 12/12 correctness and B/E timings

Part 1: v2 matches the original SQL and Neo4j in all 12 cases. Part 2: medians of 8.9 / 5.2 ms at bound 4 and 37.9 / 40.2 ms at bound 6, 30–506 times faster than the original.

7 Test Concurrent Requests

7.1 Build a Small Load Test

`loadtest.py` starts one Python process per client, each with one persistent connection. Each client warms up A-B-C once, then waits at a `multiprocessing.Barrier`. At release a common start time is set and the deadline is 15 s later; clients start cycles until the deadline, finish any started cycle and submit each next request immediately (60 s query timeout). Q4 depth 10 and Q7 length-only bound 6 (SQL v2) were tested separately with 1, 4, 8 and 16 clients, three repetitions each, one engine at a time. Every request was logged and checked after timing against reference answers, which matched across engines.

Throughput is correct completions divided by the time from barrier release to the end of the final cycle; p95 is the sorted successful latency at position [0.95n].

Workload	Clients	PG req/s	PG median ms	PG p95 ms	Neo req/s	Neo median ms	Neo p95 ms
Q4 d10	1	64.7	8.2	34.3	184.3	4.7	8.7
Q4 d10	4	161.0	12.1	63.2	938.7	3.0	8.2
Q4 d10	8	96.5	69.1	198.1	1,047.2	4.8	23.6
Q4 d10	16	92.7	99.5	475.7	978.0	10.6	50.4
Q7 b6	1	20.8	47.3	57.4	132.3	7.5	12.7
Q7 b6	4	49.7	78.4	98.1	531.8	7.7	14.1
Q7 b6	8	31.1	269.8	300.7	513.2	10.9	51.6
Q7 b6	16	30.5	506.8	828.2	410.8	25.8	89.7

Table 18: All 16 combinations (median of three repetitions); 0 errors, 0 timeouts, 0 wrong answers

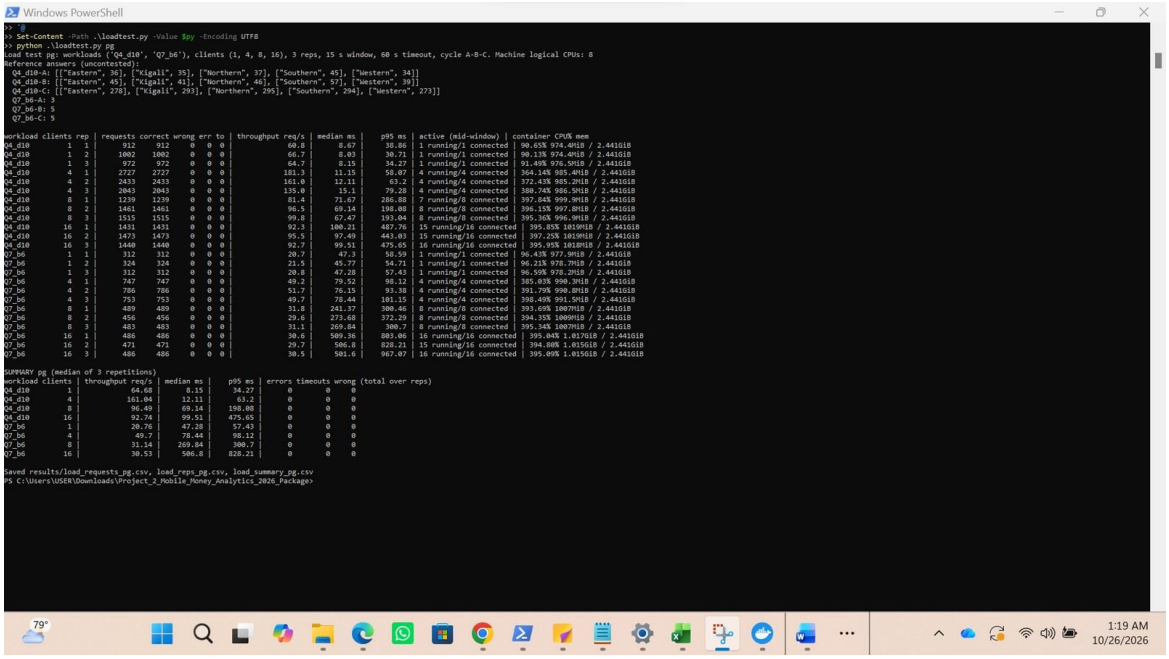


Figure 35: PostgreSQL load test: repetitions, active clients and summary

Each line is one repetition (requests, correct, wrong, errors, timeouts, throughput, median, p95, active clients and container CPU): 0 wrong and 0 errors, with CPU near 400% from 4 clients. The summary gives the median of three repetitions.

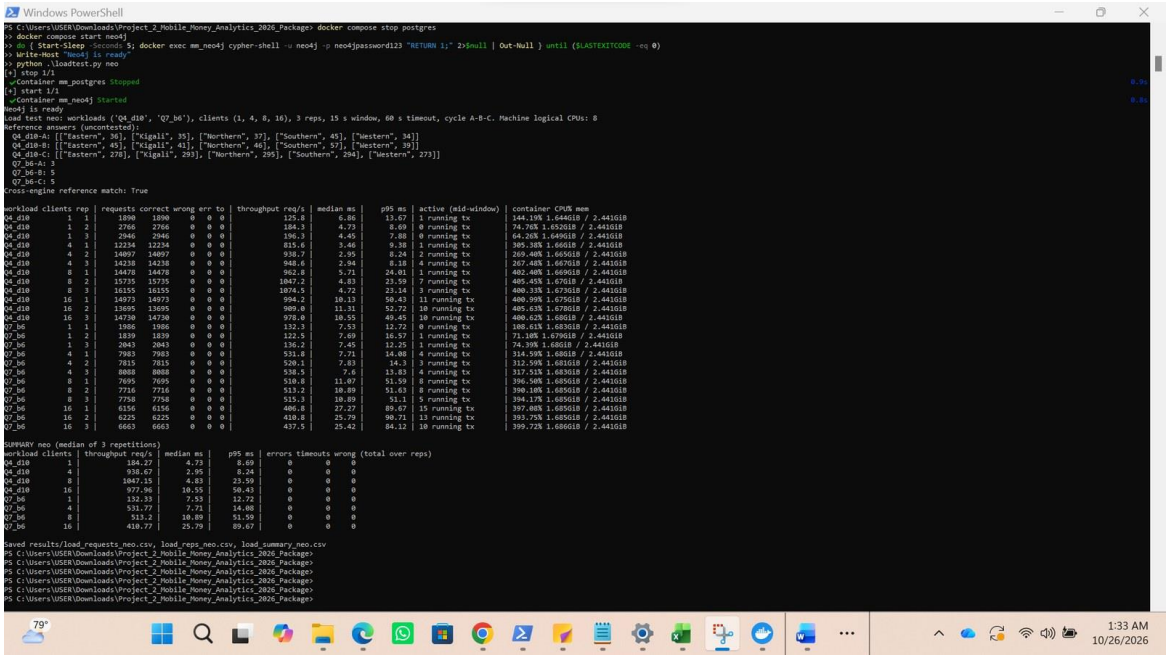


Figure 36: Neo4j load test: repetitions, active clients and summary

The same layout for Neo4j: cross-engine reference match True, 0 errors, and throughput up to 1,047 req/s for Q4 and 532 req/s for Q7.

7.2 Explain Scaling

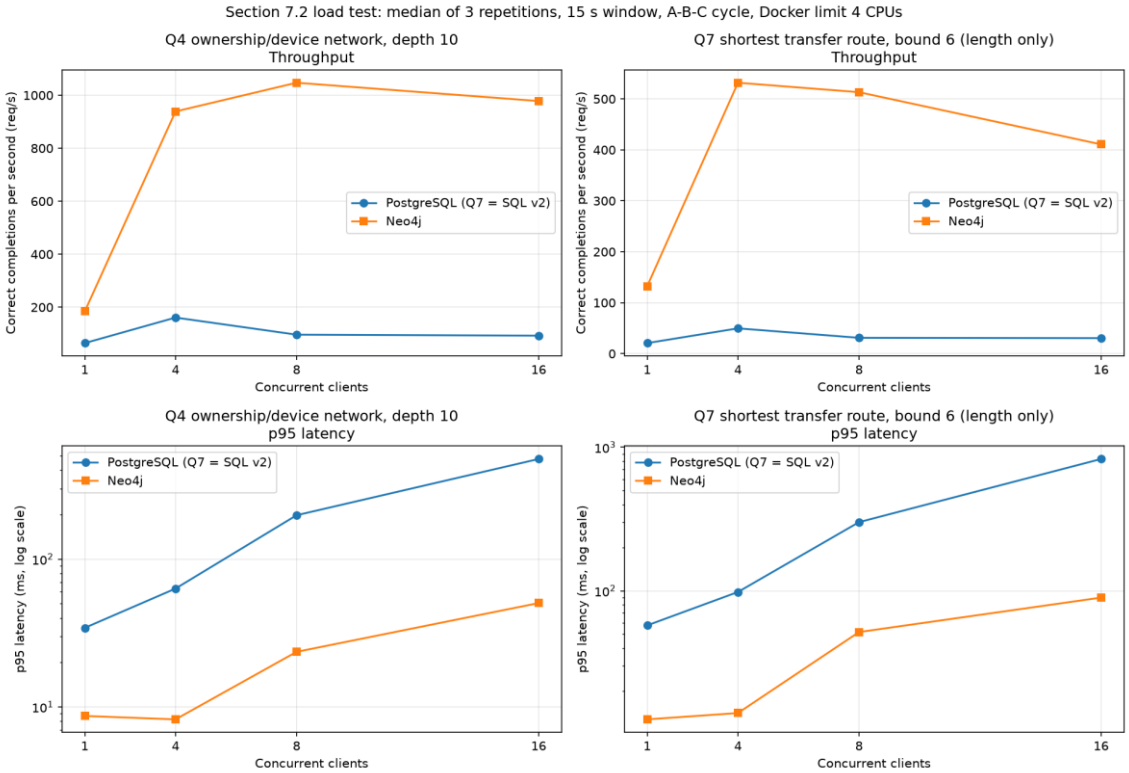


Figure 37: Throughput and p95 latency against client count

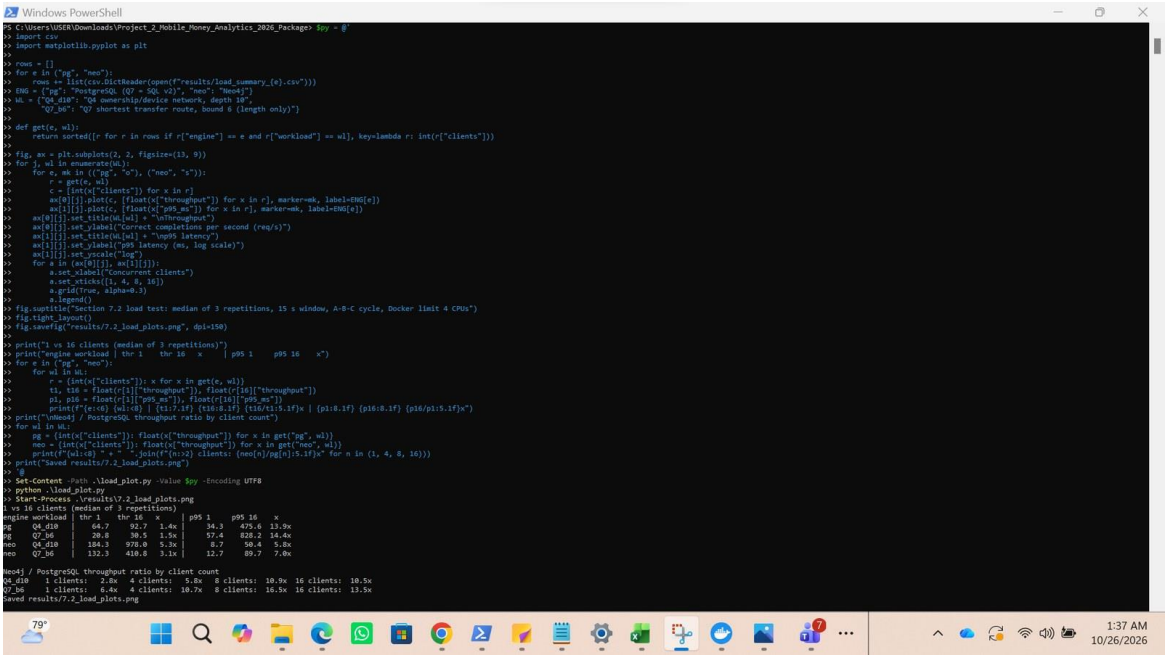


Figure 38: Plot script output: 1 versus 16 clients and Neo4j/PostgreSQL throughput ratios

The plot script prints throughput and p95 at 1 and 16 clients for each engine and the Neo4j/PostgreSQL throughput ratio at each client count (2.8 to 16.5 times).

From 1 to 16 clients, PostgreSQL's throughput rose only 1.4× (Q4) and 1.5× (Q7) while its p95 rose about 14× (Q7: 57 to 828 ms). Neo4j's throughput rose 5.3× (Q4) and 3.1× (Q7), with p95 up

6–7×. Throughput is the total correct work per second; latency is what each analyst waits. Past saturation, extra clients add little throughput but make every request slower.

Both engines saturated the Docker limit of 4 CPUs (container CPU column in Figures 35 and 36): about 400% container CPU from 4 clients for PostgreSQL and from 8 clients for Neo4j, with all reads served from memory. The plans explain the different ceilings: a PostgreSQL Q7 v2 request touches about 112,000 buffers and a Q4 request about 17,700, against about 19,500 and 4,164 Neo4j DB hits. With CPU as the shared resource, the cheaper traversal completes more requests per second. Neo4j's first repetition after a restart was slower (cold cache), which the median of three absorbs.

Does the graph advantage survive the SQL improvement? In this experiment, yes. v2 cut the single-query gap at bound 6 to 1.4–2.4×, but under load Neo4j's throughput advantage was 10.5–16.5× from 8 clients upwards. This is limited to one machine with 4 database CPUs, client processes on the same host, 15-second windows and sets A–C.

8 Make and Defend Decisions

8.1 Prioritise Investigation

Each pair is a structural "twin": nearly the same device sharing (Q3), network sizes (Q4) and Q7 lengths. What separates them is payment behaviour, so the priorities rest on Q5 and Q6.

Pair	Q3 shared devices	Q4 (d4/6/8/10)	Q5	Q6	Q7 (b4/b6)	Review first
A / D	A: handset 228001 with 8917, 16836, 24755 (days 0–29). D: handset 228301 with 5917, 13836, 21755	A 10/43/175/187; D 10/44/180/192	A 1, D 0	A 1 (len 4), D 0	3/3 both	A: money returns to it in 120 s, and every account in the loop shares A's handset
B / E	B: handset 228018 incl. 170985, 186823. E: same shape (228318)	B 46/178/188/228; E 47/183/193/230	B 1, E 0	0 both	none/5 both	B: rapid chain ending at 170985, which shares B's handset
C / F	C: handset 228043 incl. 6573, 230735. F: same shape (228343)	C 146/483/832/1,433; F 147/488/837/1,426	C 1, F 0	C 1 (len 6), F 0	none/5 both	C: chain and 6-transfer cycle in the largest network

Table 19: Pair comparison using Q3–Q7; overall order A, C, B, then D, E, F

Pair A/D in detail. On 6 September, 998 sent 200,000 RWF to 8917 at 10:00:00; 8917 sent 196,000 to 16836 at 10:00:40; 16836 sent 192,080 to 24755 at 10:01:20; and 24755 sent 188,238 back to 998 at 10:02:00. Every step was 40 s apart and exactly 98% of the previous one, and 998 ended 11,762 RWF lower. The four accounts belong to four different customers in two regions (Kigali and Northern). Each has its own main handset, yet all four were also observed on one second handset, 228001, throughout the month (7–16 observations each, fewer than on their own phones). D shares a handset in the same way, including with its destination 21755, and has the same Q7 length of 3, but shows no chain and no cycle. Device sharing alone is common here; the timed, shrinking loop is what distinguishes A. The chain and cycle evidence is in Figures 20 and 21 (Section 5); the device rows are in Figure 16 (Section 4.3).

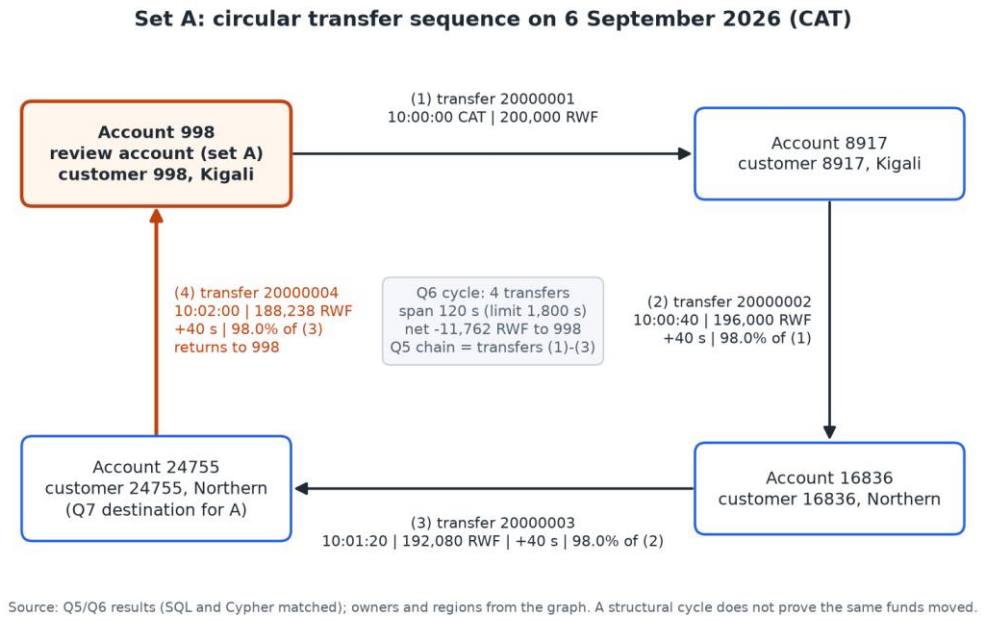


Figure 39: Transfer cycle, set A

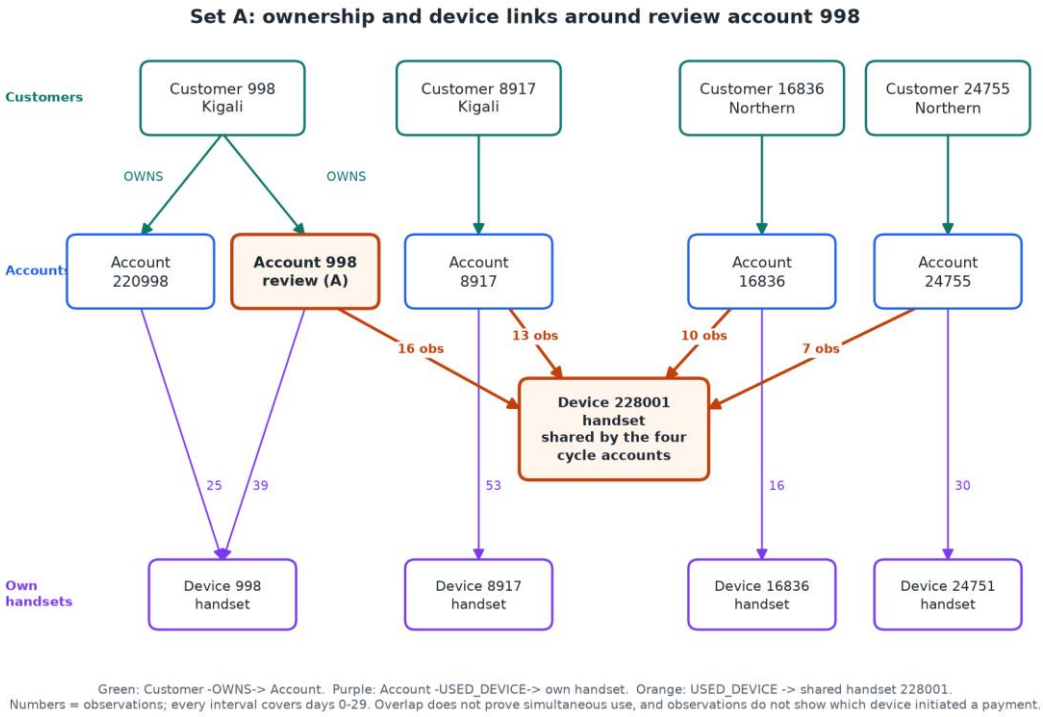


Figure 40: Ownership and device links, set A

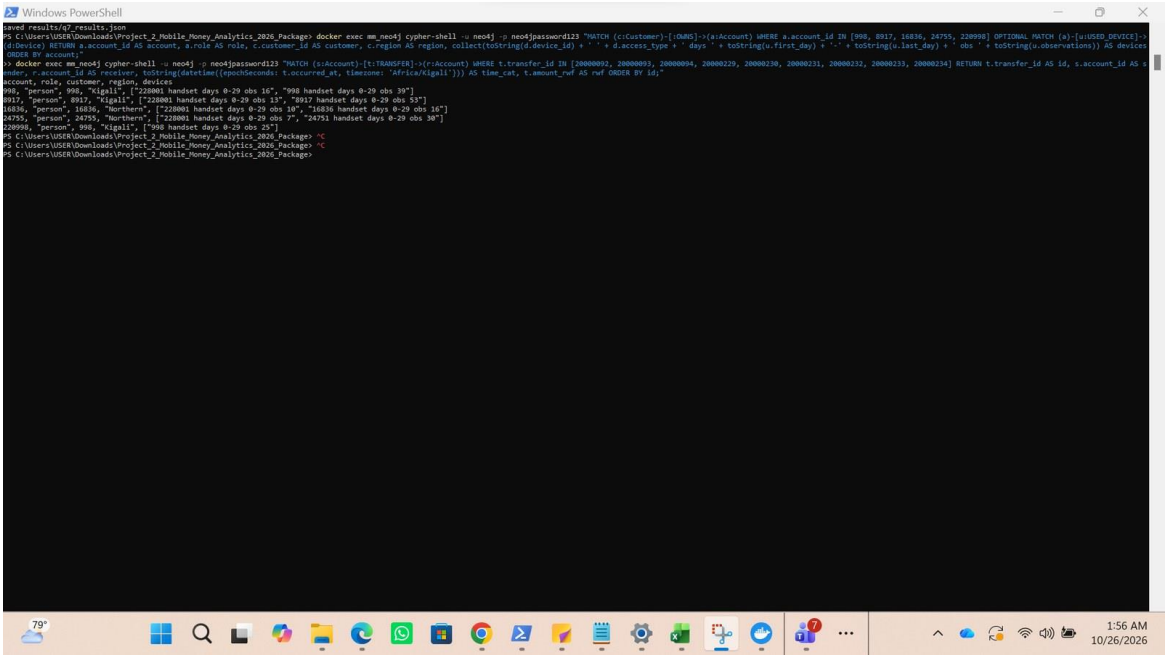


Figure 41: Owners, regions and device observations for the set A accounts

The four cycle accounts belong to four customers in Kigali and the Northern Province, each with its own main handset and some observations on shared handset 228001. Account 220998 belongs to customer 998 and uses device 998.

Ordinary explanation. Handset 228001 could belong to a relative or to a savings-group (*ikimina*) treasurer who helps family members in Kigali and the Northern Province to transact; a rotating contribution with a small amount kept or paid in fees at each step would produce a similar pattern. The link between 998 and 220998 is ordinary: both wallets belong to customer 998.

Missing evidence: which device initiated each transfer (observations do not show this), the KYC identities and relationships of the four customers, balances and cash-out events after 10:02, and history outside September. If one handset initiated all four transfers for four unrelated people, further enquiries would be justified; if they are documented members of one savings group, the concern drops. These records do not establish fraud.

8.2 Recommend a Model

We recommend **both**: PostgreSQL as the system of record for reporting and single-account checks, and Neo4j as a read-only investigation copy for multi-hop traversal.

Reporting. PostgreSQL computed the monthly daily totals (Q2) in 3.7 s against Neo4j's 47.1 s (12.6×), using a parallel scan of compact rows, while Neo4j visited about 100 million relationship records in one thread. Single-account checks (Q1, Q3, Q5, Q6) took 1.2–2.2 ms in PostgreSQL and 6.5–8.8 ms in Neo4j, where fixed per-query overhead dominates.

Traversal. Neo4j answered Q7 at bound 6 in 16–28 ms. The original recursive SQL took about 19 s, and its plan showed 427,955 depth states and 14.6 million edge probes, continuing after reaching the destination. A bidirectional rewrite returned the same 12 answers in 38–40 ms, leaving Neo4j 1.4–2.4× faster. For Q4, PostgreSQL was faster up to depth 8, and Neo4j only slightly faster at depth 10.

Concurrency. With 16 analysts, Neo4j completed 978 Q4 and 411 Q7 requests per second against PostgreSQL's 93 and 31 (10.5–13.5×), with p95 of 50–90 ms against 476–828 ms. Both saturated the 4-CPU limit; each PostgreSQL Q7 request touched about 112,000 buffers against roughly 19,500 Neo4j DB hits.

Load and storage. PostgreSQL loaded in 32.4 s plus 43.2 s for constraints and indexes and occupies 3.73 GB (1.25 GB data, 2.47 GB indexes). Neo4j imported in 109.3 s plus 44.7 s setup and occupies 2.44 GB (2.17 GB store, 0.27 GB transaction log). A graph copy adds this cost on top of the operational database.

Query and plan. The Q7 Cypher is a short SHORTEST pattern, planned as one ShortestPath operator that filters during the search. The SQL needs a recursive CTE with explicit depth and deduplication, and after improvement two searches and a join, planned as nested loops of index probes.

Modelling versus algorithm. Most of the original Q7 gap was algorithmic: bidirectional search shrank it about 500-fold. What remains is a modelling and storage benefit: relationships are stored with each node rather than found through B-tree lookups. It matters most under concurrent load.

Evidence against Neo4j. Well-written SQL comes within a few multiples for one analyst, Neo4j is 12.6× slower for reporting, and a second database adds import time, storage and synchronisation risk.

Freshness. If transfers are copied into the graph, we propose keeping it no more than 15 minutes behind PostgreSQL. A stale graph can miss a chain or cycle completed after the last refresh, so an analyst might wrongly deprioritise an account; investigation screens should show a "data as of" time, and key transfers should be confirmed in PostgreSQL.

Limits. (1) One machine with a 4-CPU, 2.4 GB Docker limit and clients on the same host. (2) One month of synthetic data and timings for sets B and E only; larger graphs or other query mixes could change the results.

8.3 Explain Individual Judgement

RWAGASANA Felicien: The Q7 SQL rewrite

Our original recursive SQL for Q7 took about 19 s for sets B and E at bound 6, against 16–28 ms in Neo4j. The EXPLAIN (ANALYZE, BUFFERS) plan showed why: the query kept (account, depth) pairs, so it produced 427,955 states, followed 14.6 million transfer edges and needed 60 million buffer hits, and it carried on to depth 6 after the destination had already appeared at depth 5. I predicted that a bidirectional search, three levels from each end, would avoid the expensive outer levels. The rewritten query returned the same lengths in all 12 cases and took 37.9 and 40.2 ms, about 500 times faster, with 111,730 buffer hits. Most of the gap therefore came from our algorithm rather than from the relational model. The finding could be overturned where both endpoints have very large neighbourhoods, or where no route exists, because both halves must then be explored completely. A check would be to time the new query on all six sets and on random account pairs, and to compare it with a simpler version that only stops at the first depth reaching the destination.

RUBAGUMYA Brave: The concurrency gap

For a single query, the improved SQL was only 1.4–2.4 times slower than Neo4j for Q7 at bound 6. Under concurrent load the picture changed: with 16 clients Neo4j completed 978 Q4 and 411 Q7 requests per second, against 93 and 31 for PostgreSQL, and PostgreSQL's Q7 p95 latency rose from 57 ms to 828 ms. Both engines reached about 400% container CPU, the 4-CPU Docker limit, and read only from memory, so CPU was the shared bottleneck. The plans explain the difference in capacity: each PostgreSQL Q7 request needs about 112,000 buffer accesses, while Neo4j needs about 19,500 DB hits, so Neo4j fits more requests into the same CPU time. Throughput stopped rising after 4 to 8 clients, while each analyst's waiting time kept growing. The finding could be overturned because the Python client processes ran on the same 8-thread laptop as the databases and competed with them for CPU. A check would be to run the clients from a second machine, give each container more CPUs, and repeat with 60-second windows to see whether the 10–16 times gap remains.

UMUHOZA Diane: Set A as first priority

I judged set A (account 998) as the first account to review. On 6 September its money went from 998 to 8917, 16836 and 24755 and back to 998 within 120 seconds, in four transfers exactly 40 seconds apart, each 98% of the previous amount (200,000 down to 188,238 RWF). The four accounts belong to four different customers in Kigali and the Northern Province, and each has its own main handset, yet all four were also observed on one shared handset, 228001, for the whole month. Set D has a very similar device structure and the same Q7 length of 3, but no chain or cycle, which shows that device sharing alone is common and not decisive. The combination of a fast, regular, shrinking loop and a shared second phone is what places A above the others. This judgement could be overturned by evidence the data does not contain: which device initiated each transfer, KYC records showing that the four customers are one family or savings group, and the same rotation repeating every month. If those show an ordinary arrangement, A should drop in priority.

Member	ID	Contributions
RWAGASANA Felicien	101364	Capture, PostgreSQL schema and load, Q7 SQL improvement (6.3), README
RUBAGUMYA BRAVE	101392	Neo4j import, timing and plans (6.2), concurrent load test (7)
UMUHOZA Diane	101286	Queries Q1–Q6, 61-case comparison (6.1), investigation priorities (8.1)
All members		Model recommendation (8.2), report writing and review

Table 20: Member contributions

LLM and external assistance. Claude (Anthropic) was used to help draft PowerShell and Python commands (plan capture, the Q7 bidirectional query, the load-test, plotting and diagram scripts), to explain query plans and to help structure this report. Every output was checked by running it ourselves: the Q7 rewrite against the original SQL and Neo4j on all 12 cases (12/12), the load-test answers against reference answers (0 wrong, cross-engine match), and the full 61-case comparison rerun (61/61). All numbers come from our own result files.

9 Submit Reproducible Work

9.2 Provide Code and Verification

The repository and the matching ZIP contain the generator, capture and import programs, definitions, queries, checks, benchmarks, raw results and plans, with a README listing the commands in execution order and placeholder secrets (`.env.example`; real passwords are excluded by `.gitignore`). CSVs and database stores are kept local. The generator is unchanged: its SHA256 (`2EA0E59D…7E9455A`) matches the copy in the original package.

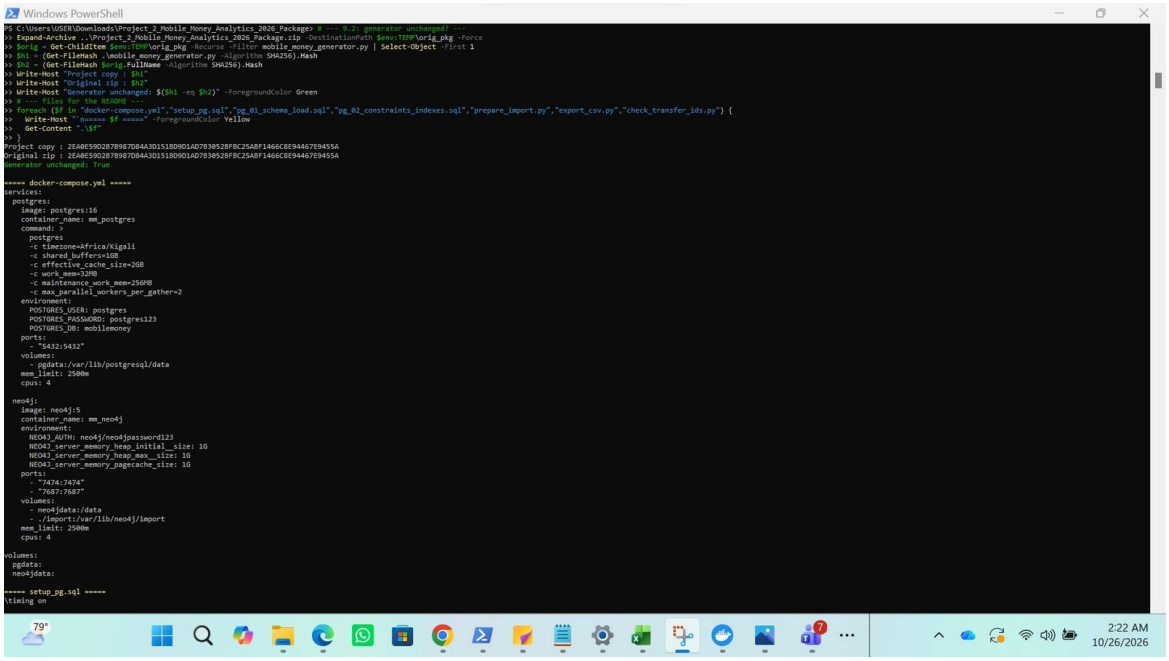


Figure 42: Generator unchanged: SHA256 matches the original package

The SHA256 of `mobile_money_generator.py` equals that of the copy in the original package zip (Generator unchanged: True).

The rerun below repeats the Section 6.1 check (Figure 23) on the submitted code.

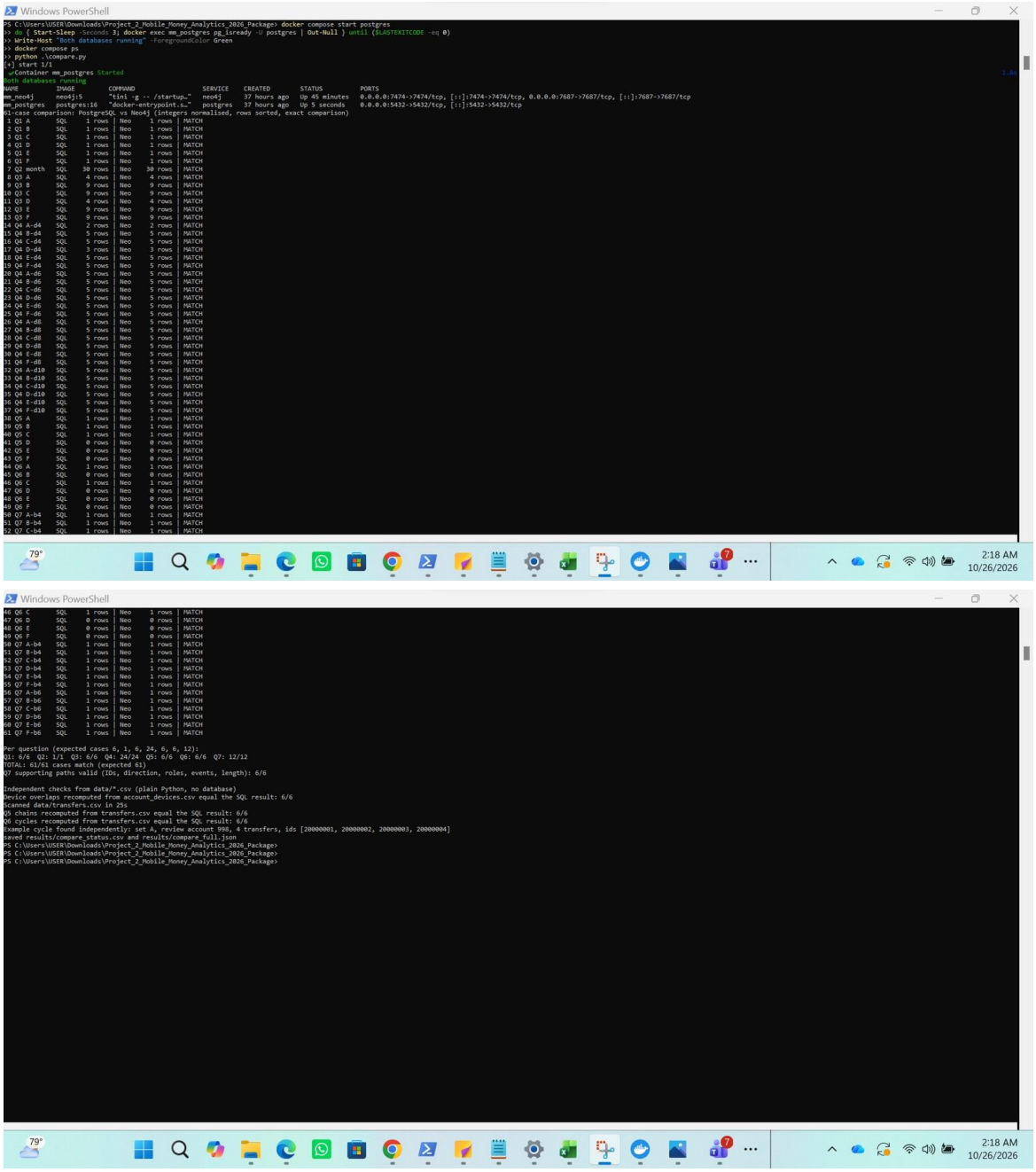


Figure 43: Submitted 61-case comparison rerun: 61/61, paths 6/6, independent checks

Rerun of `compare.py` on the submitted code: 61/61 cases, 6/6 Q7 paths and all independent checks pass.

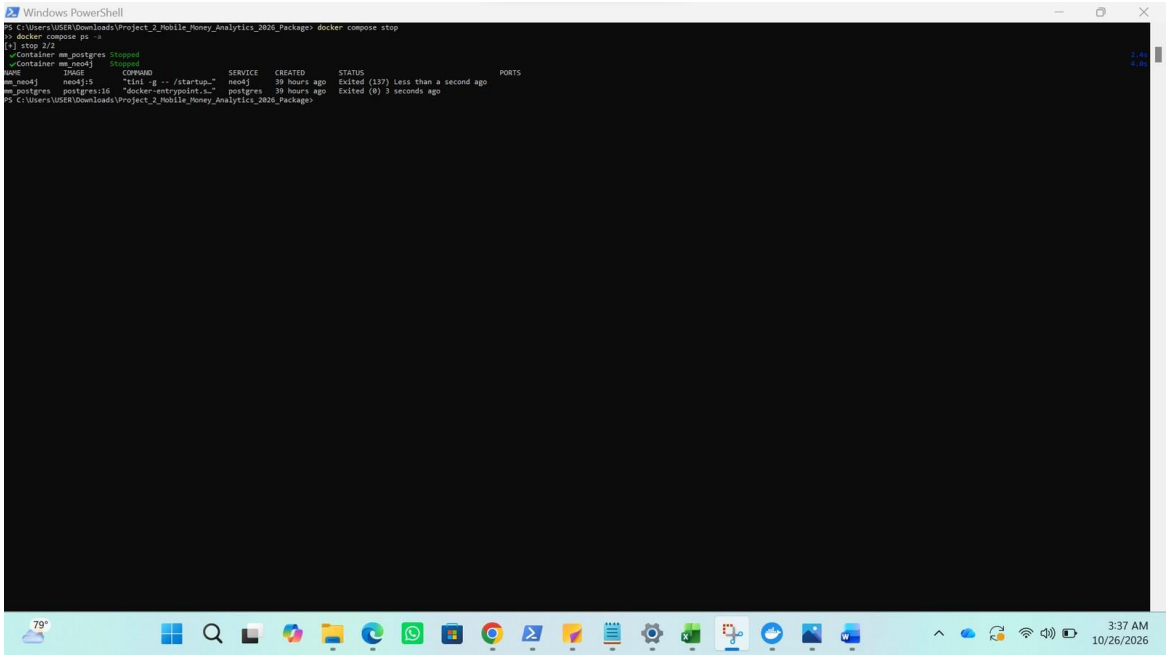


Figure 44: Project database services stopped

`docker compose stop` stops both project services, and `docker compose ps -a` lists `mm_postgres` and `mm_neo4j` as Exited. The database stores stay in the Docker volumes `pgdata` and `neo4jdata`, so they are retained for later runs.